Chương 5
Giao dịch
(Transaction)

GVHD: TS. Phan Thị Thể

FIT – HCM-UTE

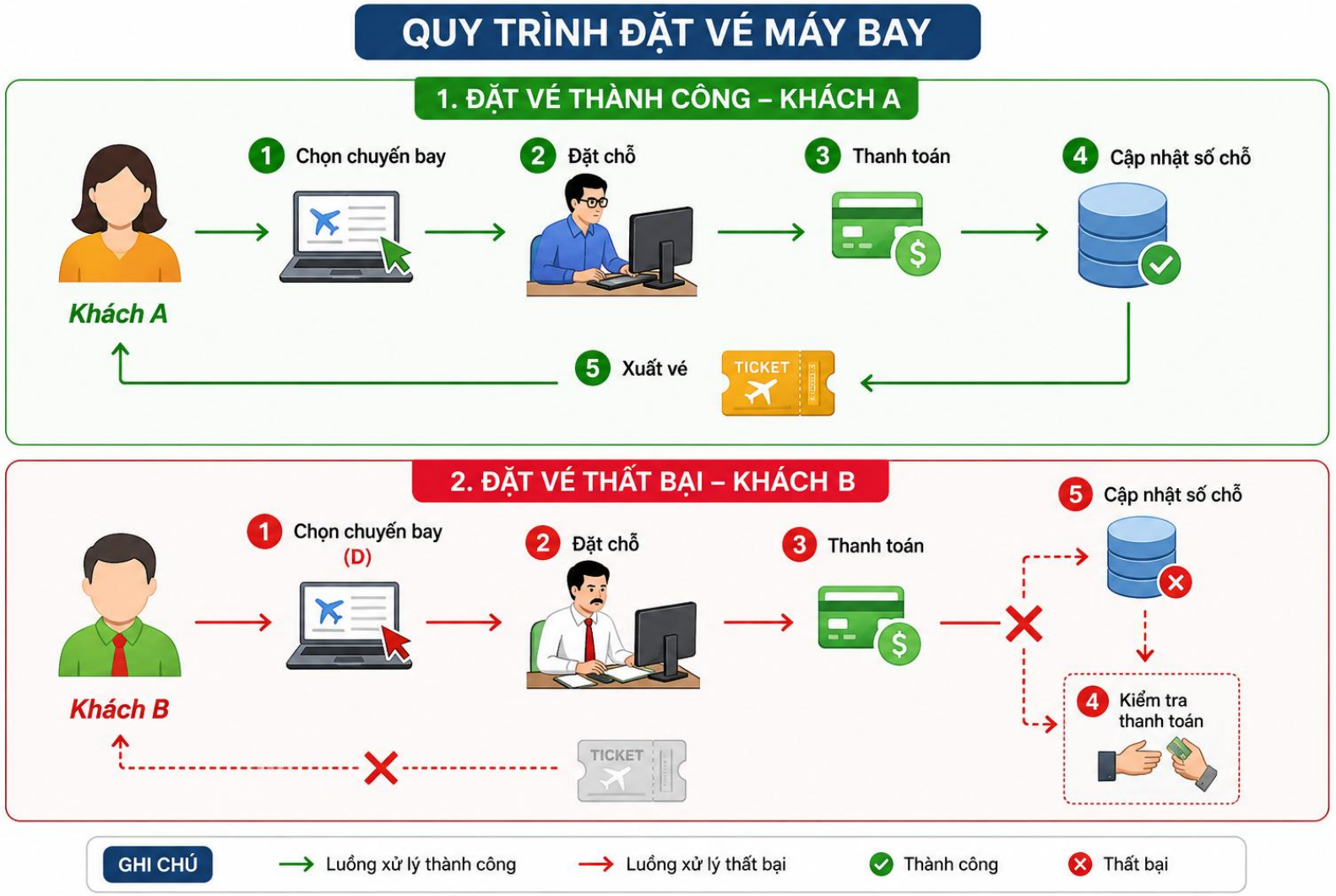
Email: thept@hcmute.edu.vn

Nội dung chương 5

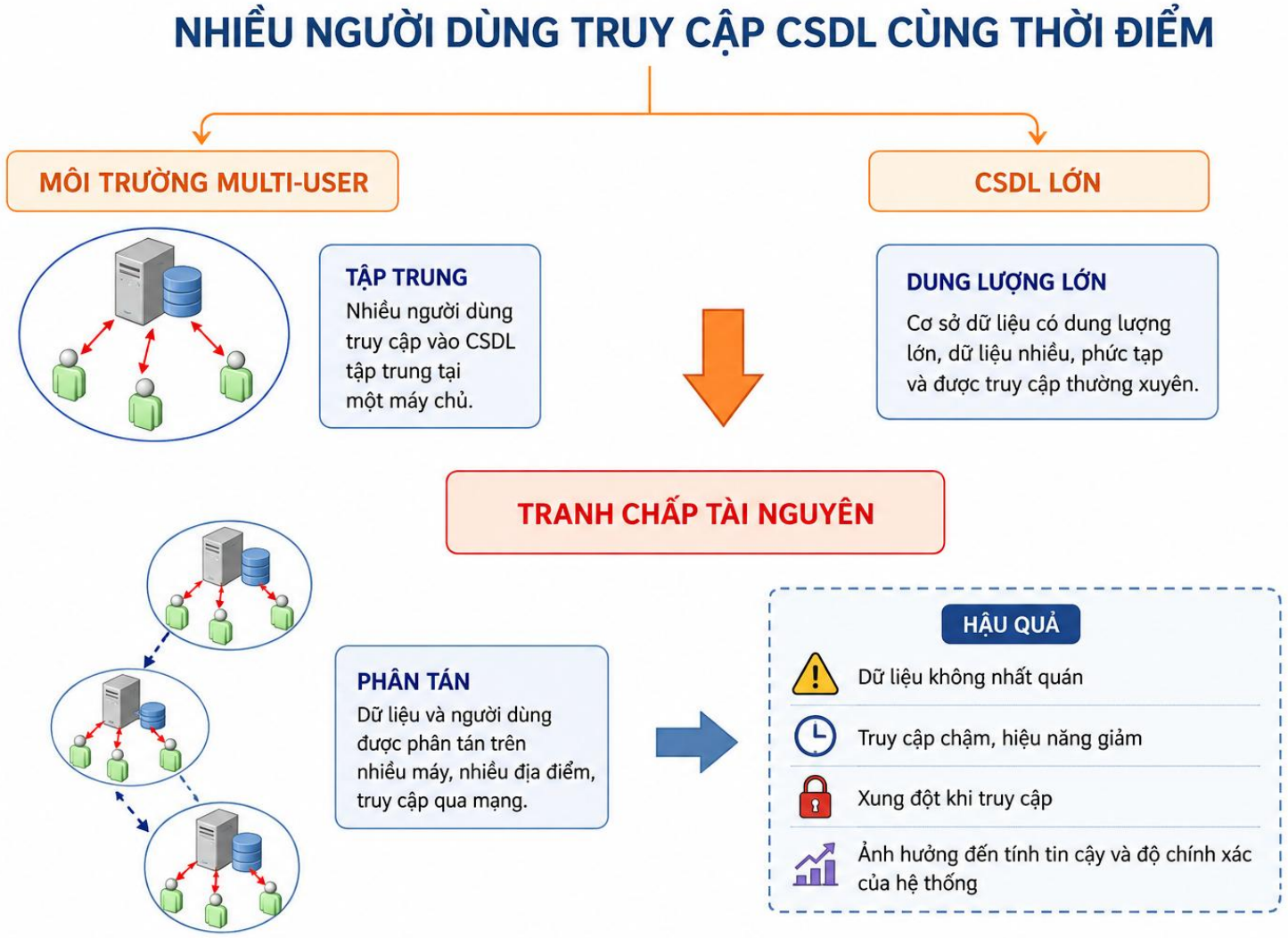
1. Giới thiệu về Giao dịch (Transaction).
2. Lịch trình
3. Giao dịch trong SQL
4. Bài tập

1. Giới thiệu về Giao dịch (Transaction).

1. Giới thiệu về Giao dịch (Transaction)



1. Giới thiệu về Giao dịch (Transaction)

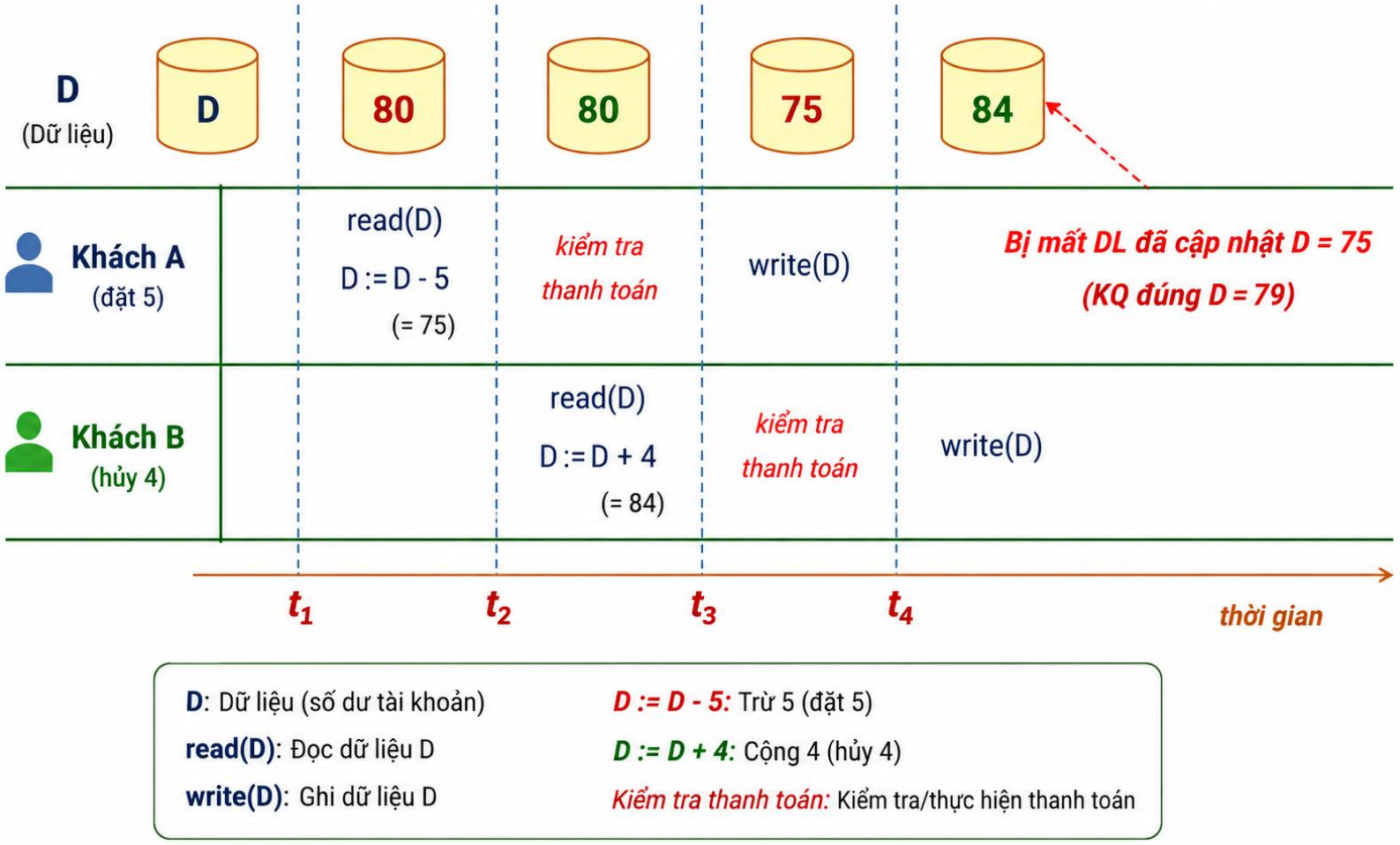


1. Giới thiệu về Giao dịch (Transaction)

- 1 cách phân loại HQT CSDL khác là dựa trên số lượng người sử dụng đồng thời:
  - Single-user
  - Multi-user: hầu hết các HQT CSDL
- Xử lý xen kẽ (Interleaved processing)
- Xử lý song song (Parallel processing)
- **Từ đó đặt ra vấn đề:**
  - Mất dữ liệu đã cập nhật (Lost Update Problem)
  - Số liệu tổng hợp không chính xác (Incorrect Summary Problem)
  - Khai thác dữ liệu 'giả' (Dirty Read Problem)

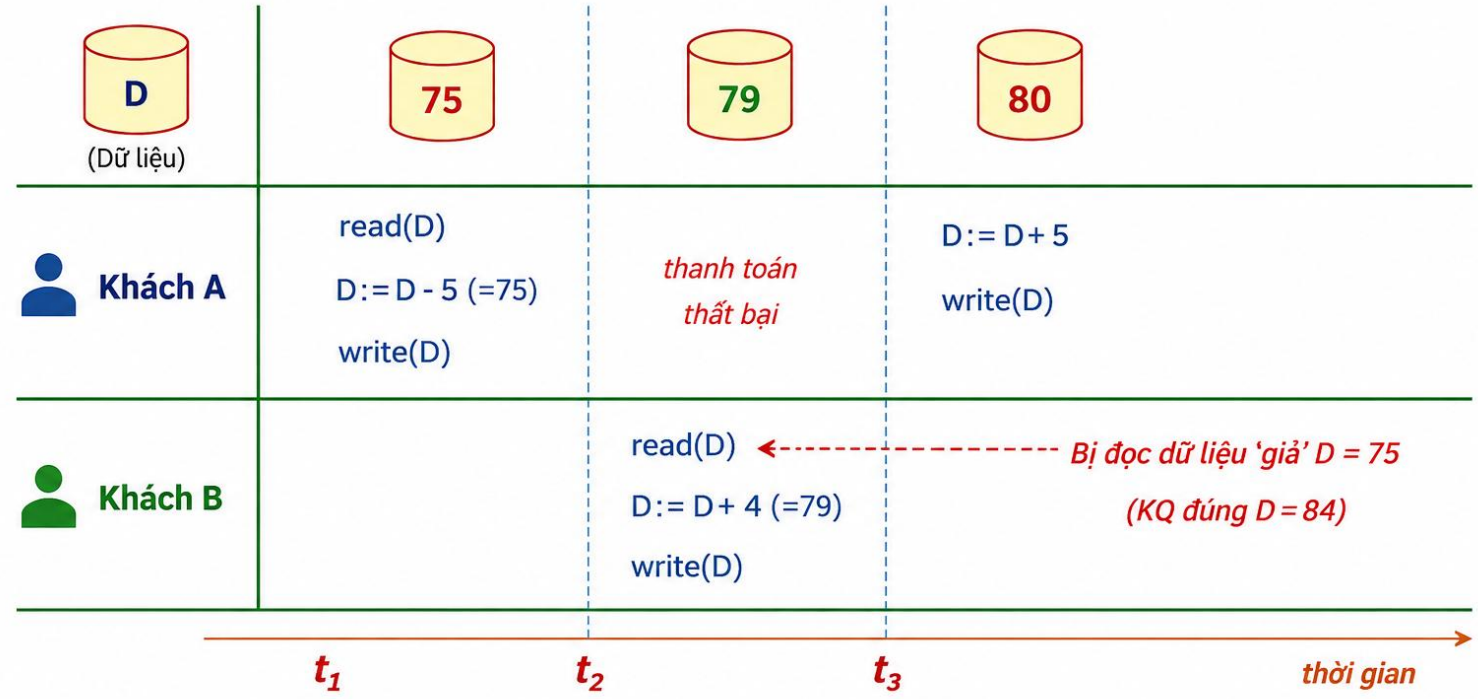
1. Giới thiệu về Giao dịch (Transaction)

- Trường hợp 1: Mất dữ liệu đã cập nhật (Số chỗ còn D=80)



1. Giới thiệu về Giao dịch (Transaction)

- Trường hợp 2: Khai thác dữ liệu giả (Số chỗ còn D=80)



D: Dữ liệu (số dư tài khoản)

read(D): Đọc dữ liệu D

write(D): Ghi dữ liệu D

D := D - 5: Trừ 5 (đặt 5)

D := D + 4: Cộng 4 (hủy 4)

Tham khảo: Khách B đọc giá trị 75 là dữ liệu chưa được ghi chính thức (*dirty read*). Kết quả đúng phải là 84.

1. Giới thiệu về Giao dịch (Transaction)

- Trường hợp 3: Số liệu tổng hợp không chính xác
  - W1, W2,…, Wn là số chỗ đã đặt ở trạm 1, trạm 2, …, trạm n
  - Tính tổng số vé đã đặt của tất cả các trạm: $Z_n = W_1 + W_2 + \ldots + W_n$
  - Tính được tới trạm 4: $Z_4 = W_1 + W_2 + W_3 + W_4$
  - Khi đó, trạm 1 đặt thêm 3 chỗ và trạm 2 trả lại 1 chỗ
  - Tại trạm n: $Z_n = Z_4 + W_5 + \cdots + W_n$

$\Longrightarrow$ Zn không còn chính xác

- Trong cả ba trường hợp, đặt ra yêu cầu một DBMS hỗ trợ chức năng giải quyết tranh chấp có tính **đúng đắn, ổn định, an toàn, có thời gian đáp ứng** $\rightarrow$ **GIAO DỊCH**

1. Giới thiệu về Giao dịch (Transaction)

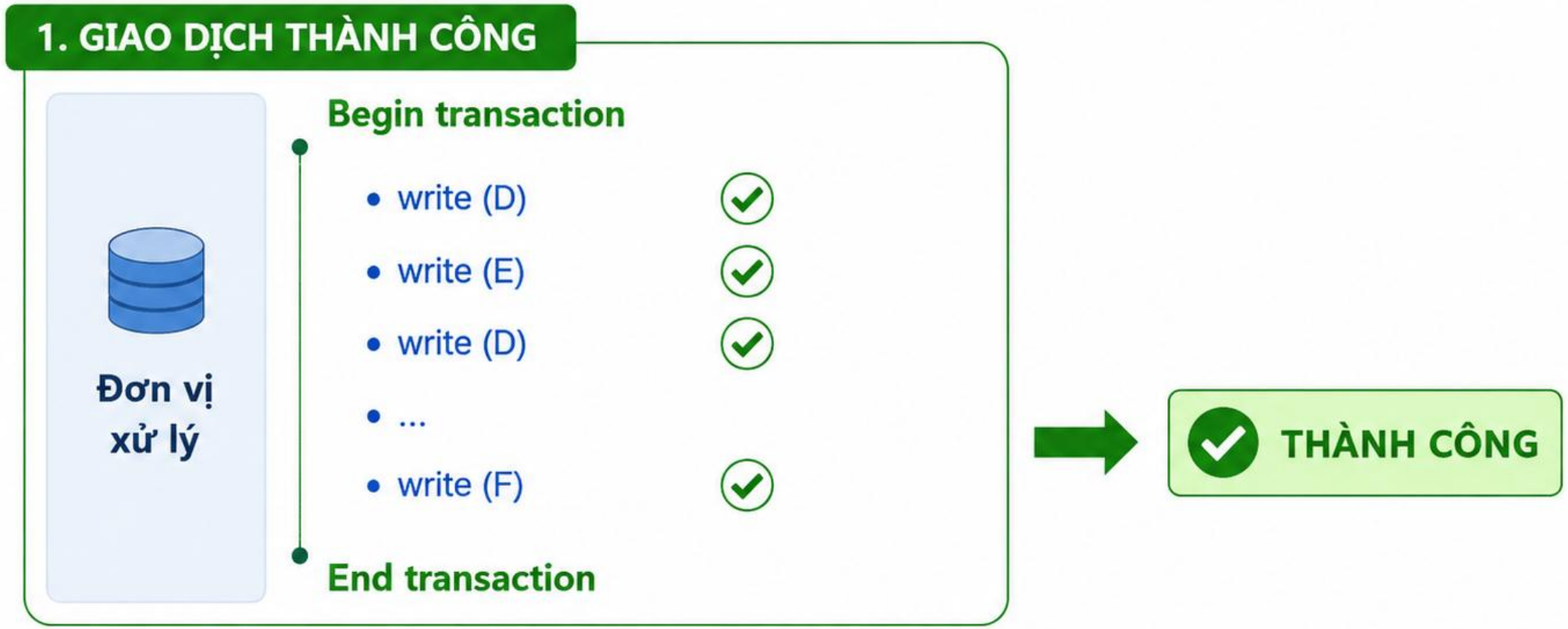
Khái niệm Giao dịch (Giao tác)

- Là một chương trình được thực thi đóng vai trò như một đơn vị xử lý truy xuất CSDL.
- Một giao dịch có thể gồm những xử lý trên 1 hoặc nhiều CSDL.
- Giao dịch có thể được viết trong ngôn ngữ SQL hoặc một ngôn ngữ lập trình nào đó.

- Thành phần của giao dịch:
  - Begin transaction: bắt đầu giao tác
  - Read/Write: các thao tác đọc/ghi trong giao tác
  - End transaction: kết thúc giao tác
  - Commit: kết thúc thành công giao tác → Dữ liệu sẽ được ghi nhận trên CSDL
  - Rollback/Abort: giao tác bị thất bại → Khi có 1 thao tác trong giao tác không hoàn tất, trở về trạng thái trước khi giao tác bắt đầu.

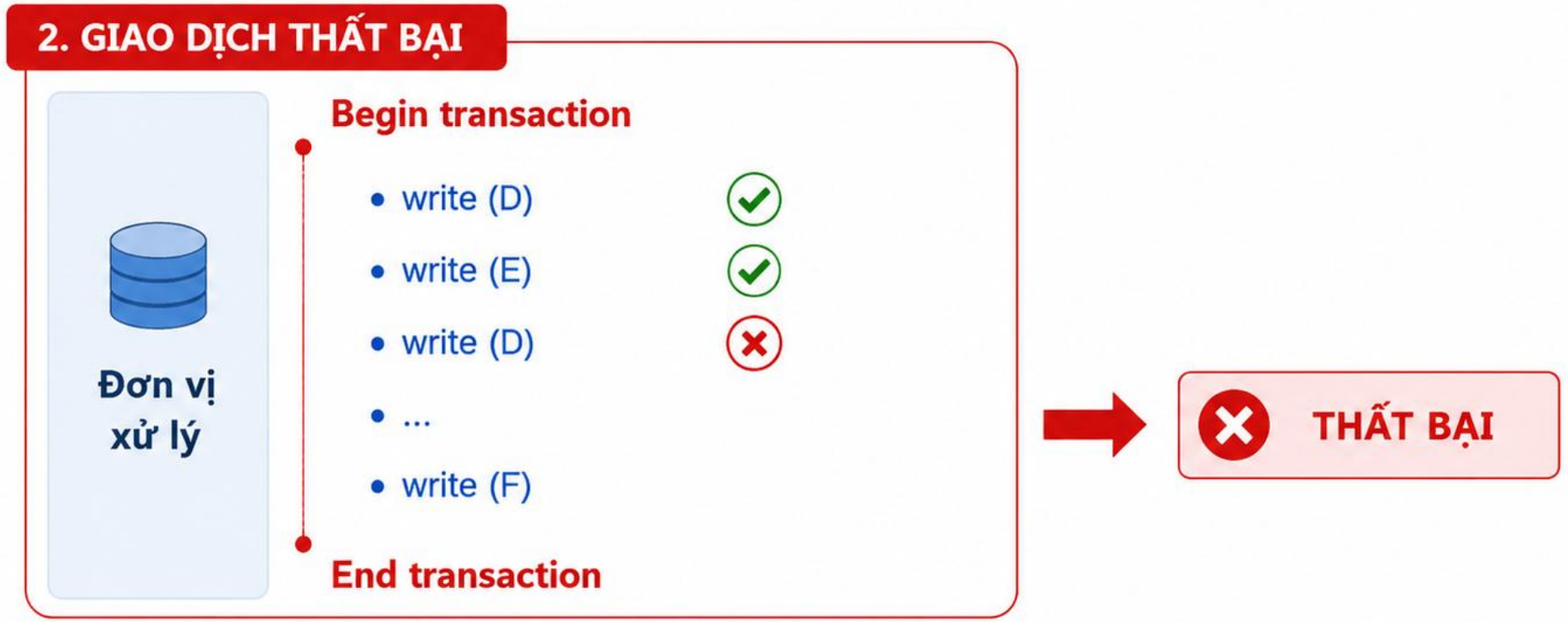
1. Giới thiệu về Giao dịch (Transaction)

Khái niệm Giao dịch (Giao tác)



1. Giới thiệu về Giao dịch (Transaction)

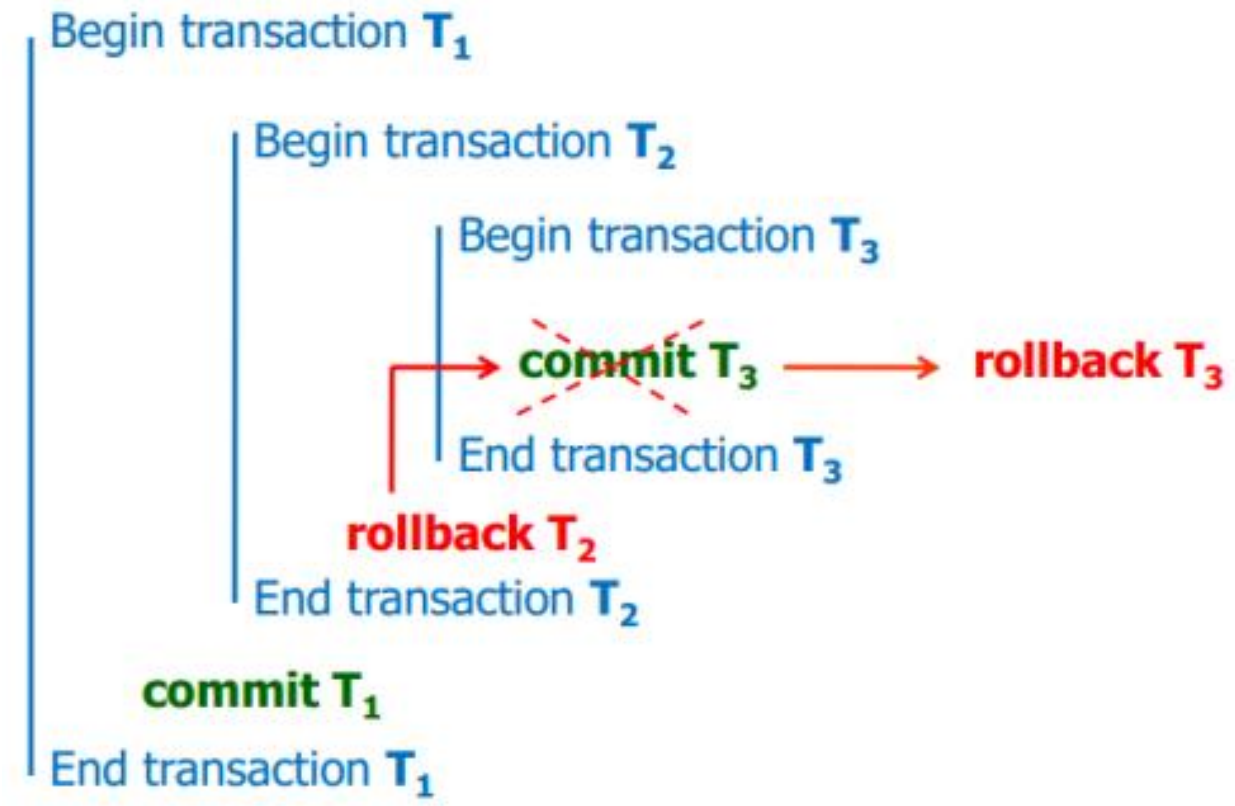
Khái niệm Giao dịch (Giao tác)



1. Giới thiệu về Giao dịch (Transaction)

Khái niệm Giao dịch (Giao tác)

- Thực hiện các giao tác lồng nhau



1. Giới thiệu về Giao dịch (Transaction)

Khái niệm Giao dịch (Giao tác)

- Chọn chuyến bay

Begin transaction

- Khóa đặt chỗ chuyến bay
- Đặt chỗ

If (thanh toán thành công)

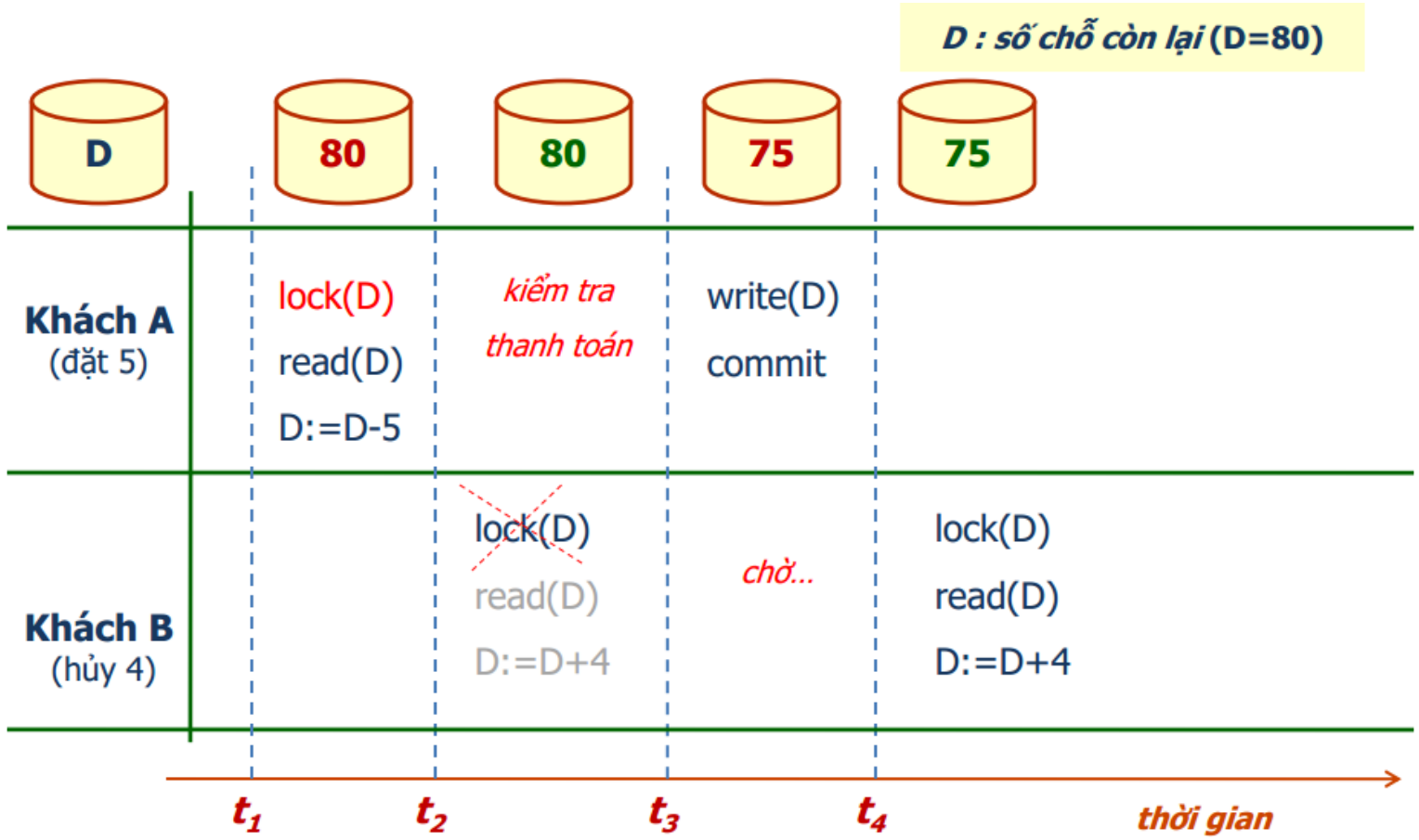
commit

Else

rollback

- Mở khóa đặt chỗ chuyến bay

End transaction



1. Giới thiệu về Giao dịch (Transaction)

- Để đảm bảo tính toàn vẹn của dữ liệu, phải đảm bảo các tính chất của giao dịch:
  - Tính nguyên tố (**A**tomicity): hoặc tất cả các xử lý trong giao tác được hoàn tất hoặc không có bất kỳ xử lý nào được thực hiện
  - Tính nhất quán (**C**onsistency): Khi một giao dịch thành công, phải đảm bảo sự nhất quán của CSDL trước và sau khi xảy ra giao dịch.
  - Tính cô lập (**I**solation): một giao tác khi thực hiện sẽ độc lập với các giao tác khác đang thực hiện đồng thời.
  - Tính bền vững (**D**urability): mọi thay đổi về dữ liệu khi giao tác hoàn tất phải được ghi nhận bền vững trên CSDL. Sau khi một giao dịch thành công, các thay đổi với CSDL phải còn nguyên cho dù có xảy ra sự cố hệ thống.

1. Giới thiệu về Giao dịch (Transaction)

Công việc của Hệ quản trị CSDL khi xử lý giao dịch

- **ĐỌC/ GHI DỮ LIỆU**
  - CSDL nằm trên đĩa.
  - Các truy xuất CSDL gồm:
    - Đọc nội dung X: lấy nội dung X từ CSDL trên đĩa ghi vào vùng nhớ đệm của giao dịch.
    - Ghi nội dung X: chép nội dung X từ vùng nhớ đệm của giao dịch vào lại CSDL trên đĩa.
  - Thực tế: Write không nhất thiết là thực hiện việc ghi vào CSDL trên đĩa mà có thể ghi tạm trên bộ nhớ và ghi vào đĩa muộn hơn.

1. Giới thiệu về Giao dịch (Transaction)

Công việc của Hệ quản trị CSDL khi xử lý giao dịch

- Ví dụ: T là một giao dịch chuyển 50 từ Tài khoản A sang B.

```
Read(A);
A=A-50;
Write(A);
Read(B);
B=B+50;
Write(B);
```

1. Giới thiệu về Giao dịch (Transaction)

ACID

- **Tính nguyên tử** (A): Sự cố xảy ra sau Write(A) và trước Read(B) → ?
- **Tính nhất quán** (C): Tổng A và B là không đổi khi thực hiện T (Tiền không tạo ra hoặc mất đi khi thực hiện giao dịch)
- Tồn tại thời điểm hệ thống ở trạng thái không nhất quán → cần phải có tính nguyên tử.
- **Tính cô lập** (I): ngay cả khi A và C được đảm bảo cho mỗi giao dịch, trạng thái không nhất quán vẫn có thể xảy ra khi một số giao dịch xảy ra đồng thời.
- VD: tại thời điểm không nhất quán tạm thời khi thực hiện T →1 giao dịch khác xảy ra.
- Xử lý đơn giản: thực hiện tuần tự các giao dịch → giảm hiệu năng hệ thống.

1. Giới thiệu về Giao dịch (Transaction)

ACID

- **Tính bền vững** (D): giả sử xảy ra sự cố mất dữ liệu trên bộ nhớ, dữ liệu trên đĩa không mất.
- Đảm bảo tính bền vững:
  - Đảm bảo các thay đổi đã được viết trên đĩa trước khi giao dịch kết thúc.
  - Thông tin về những thay đổi do giao dịch được viết trên đĩa đủ để xây dựng lại CSDL khi hệ thống khởi động lại sau sự cố.

1. Giới thiệu về Giao dịch (Transaction)

- **Thành phần của HQT CSDL**
  - Đảm bảo tính nguyên tử: thành phần quản trị giao dịch (transaction management component).
  - Đảm bảo tính bền vững: thành phần quản trị phục hồi (recovery management component).
  - Đảm bảo tính cô lập: thành phần quản trị cạnh tranh (concurrency control component)

1. Giới thiệu về Giao dịch (Transaction)

- **Tình huống như sau: Có 2 giao dịch T1 và T2**

(a)

T_1

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);

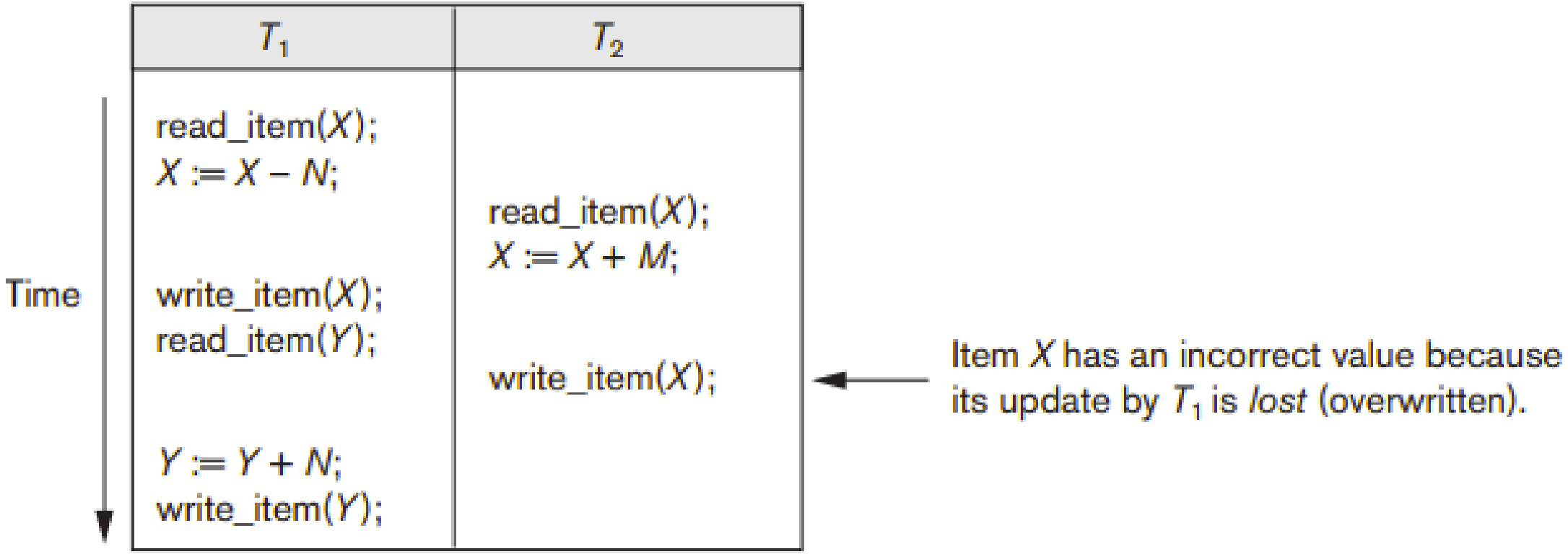
(b)

T_2

read_item(X);
$X := X + M$;
write_item(X);

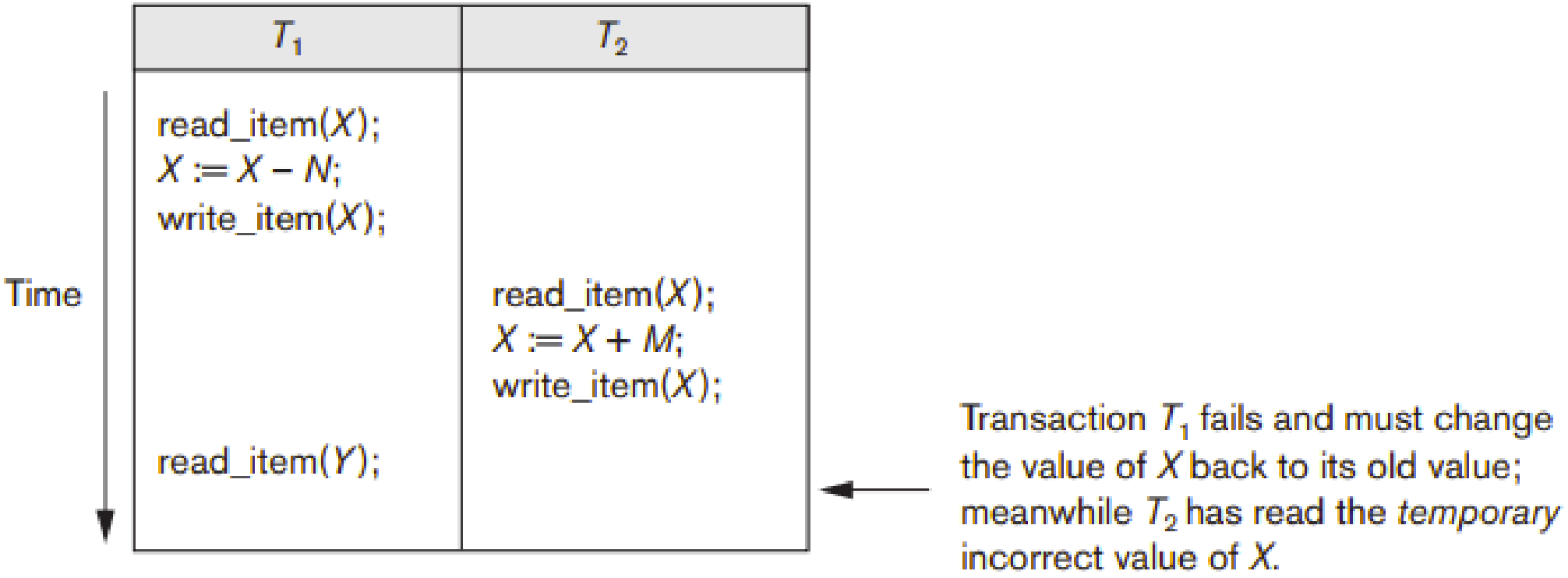
1. Giới thiệu về Giao dịch (Transaction)

- **Sự cần thiết của thành phần quản trị cạnh tranh**



1. Giới thiệu về Giao dịch (Transaction)

- **Sự cần thiết của thành phần quản trị cạnh tranh**



1. Giới thiệu về Giao dịch (Transaction)

- **Sự cần thiết của thành phần quản trị cạnh tranh**

T_1	T_3

	sum := 0;
	read_item(*A*);
	sum := *sum* + *A*;
	. . .

read_item(*X*);
X := *X* − *N*;
write_item(*X*);	

	read_item(*X*);
	sum := *sum* + *X*;
	read_item(*Y*);
	sum := *sum* + *Y*;

read_item(*Y*);
Y := *Y* + *N*;
write_item(*Y*);	

← T_3 reads *X* after *N* is subtracted and reads *Y* before *N* is added; a wrong summary is the result (off by *N*).

1. Giới thiệu về Giao dịch (Transaction)

- **Thành phần quản trị phục hồi đơn giản**
  - Điều kiện: 1 giao dịch hoạt động ở 1 thời điểm và thực hiện trên 1 bản sao của CSDL thật.
  - CSDL thật là file trên đĩa. Một con trỏ db_pointer lưu trên đĩa và trỏ đến bản sao hiện hành của CSDL đó.
  - Một giao dịch muốn cập nhật CSDL, đầu tiên tạo ra một bản sao đầy đủ của CSDL.Nếu giao dịch bị bỏ dở, bản sao mới bị xoá. Bản sao cũ của CSDL không bị ảnh hưởng.

1. Giới thiệu về Giao dịch (Transaction)

- **Thành phần quản trị phục hồi đơn giản**
  - Nếu giao dịch hoàn tất, nó được được bàn giao:
    - Đảm bảo rằng tất cả các trang của bản sao mới đã được viết lên đĩa.
    - Con trỏ được cập nhật để trỏ đến bản sao mới; bản sao mới trở thành bản sao hiện hành của CSDL.
    - Bản sao cũ bị xoá đi. Giao dịch được gọi là đã được được bàn giao (committed) tại thời điểm sự cập nhật con trỏ db_pointer được ghi lên đĩa.

1. Giới thiệu về Giao dịch (Transaction)

- **Thành phần quản trị phục hồi đơn giản**
- Sự cố giao dịch:
  - Nếu giao dịch thất bại tại thời điểm bất kỳ trước khi con trỏ db_pointer được cập nhật, nội dung cũ của CSDL không bị ảnh hưởng. Ta có thể bỏ dở giao dịch bởi xoá bản sao mới.
  - Mỗi khi giao dịch được được bàn giao (committed), tất cả các cập nhật mà nó đã thực hiện là ở trong CSDL được trỏ bởi db_pointer.

1. Giới thiệu về Giao dịch (Transaction)

- **Thành phần quản trị phục hồi đơn giản**
- **Sự cố giao dịch:**
  - Nếu giao dịch thất bại tại thời điểm bất kỳ trước khi con trỏ db_pointer được cập nhật, nội dung cũ của CSDL không bị ảnh hưởng. Ta có thể bỏ dở giao dịch bởi xoá bản sao mới.
  - Mỗi khi giao dịch được được bàn giao (committed), tất cả các cập nhật mà nó đã thực hiện là ở trong CSDL được trỏ bởi db_pointer.

1. Giới thiệu về Giao dịch (Transaction)

- **Thành phần quản trị phục hồi đơn giản**
- **Sự cố hệ thống:**
  - Sự cố hệ thống xảy ra tại thời điểm trước khi db_pointer đã được cập nhật được viết lên đĩa. Khi đó, khi hệ thống khởi động lại, nó sẽ đọc db_pointer và như vậy sẽ thấy nội dung gốc của CSDL – không hiệu quả nào của giao dịch được nhìn thấy trên CSDL.
  - Sự cố hệ thống xảy ra sau khi db_pointer đã được cập nhật lên đĩa: Khi hệ thống khởi động lại, nó sẽ đọc db_pointer và sẽ thấy nội dung của CSDL sau tất cả các cập nhật đã thực hiện bởi giao dịch.

1. Giới thiệu về Giao dịch (Transaction)

- **Thành phần quản trị phục hồi đơn giản**
- Việc ghi db_pointer phải là nguyên tử.
- Sự thực thi này cực kỳ thiếu hiệu quả trong ngữ cảnh CSDL lớn, do sự thực hiện một giao dịch đòi hỏi phải sao toàn bộ CSDL.
- Sự thực thi này không cho phép các giao dịch thực hiện đồng thời với các giao dịch khác.

1. Giới thiệu về Giao dịch (Transaction)

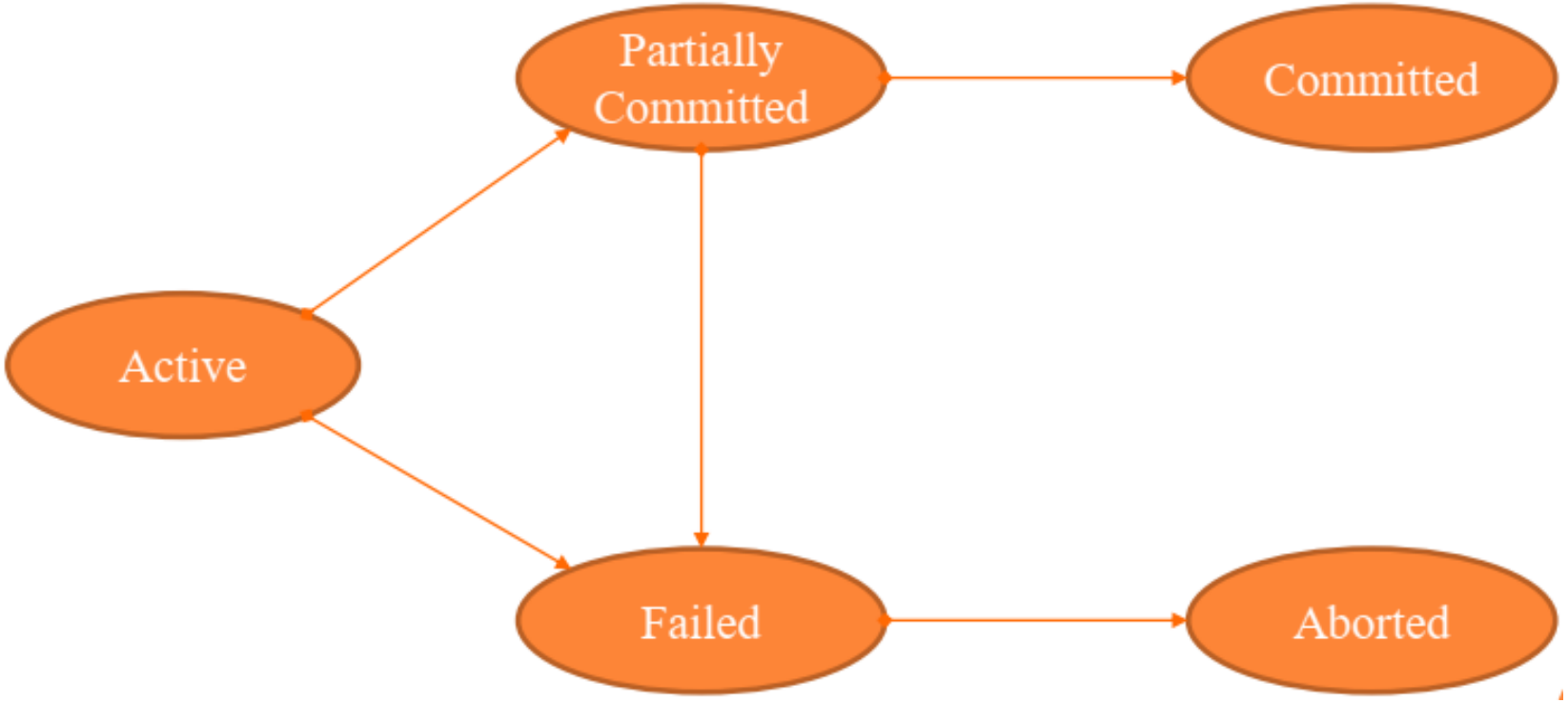
- **Trạng thái giao dịch**
- Trên thực tế, một giao dịch có thể không hoàn tất công việc → Giao dịch không hoàn thành (Uncommited).
- Tính nguyên tử: Giao dịch bị bỏ dở này không thay đổi trạng thái CSDL → tất cả các thay đổi phải được rollback.
- Một giao dịch hoàn tất gọi là được bàn giao (committed) và không thể hủy bỏ những thay đổi được thực hiện bằng committed transaction.
- Nếu muốn thay đổi: thực hiện giao dịch bù (compensating transaction) thuộc về người sử dụng.

1. Giới thiệu về Giao dịch (Transaction)

- **Các trạng thái giao dịch**
- Hoạt động (Active): trạng thái khởi đầu
- Được bàn giao bộ phận (Partially committed): khi lệnh cuối cùng được thực hiện.
- Thất bai (Failed)
- Bỏ dở (aborted)
- Được bàn giao (Committed): giao dịch thành công.

1. Giới thiệu về Giao dịch (Transaction)

- **Các trạng thái giao dịch**
- Giao dịch kết thúc: Committed hoặc Aborted.



1. Giới thiệu về Giao dịch (Transaction)

Xử lý giao dịch đồng thời

- Khó khăn trong vấn đề đảm bảo sự nhất quán dữ liệu.
- Cách đơn giản nhất: thực hiện tuần tự.
- Một giao dịch gồm nhiều bước:
  - I/O
  - Xử lý trên CPU
- Tăng hiệu suất sử dụng CPU và đĩa.
- Tăng số lượng giao dịch có thể xử lý trong một đơn vị thời gian.
- Một số giao dịch khác biệt về thời gian thực hiện.
- Thực hiện tuần tự: chờ đợi quá trình dài hoàn tất…
- Để đảm bảo tính nhất quán: hệ QT CSDL phải điều
- khiển sự trao đổi giữa các giao dịch đồng thời – sơ đồ
- điều khiển cạnh tranh (concurrency control scheme)

2. Lịch trình

2.1. Khái niệm
2.2. Lịch trình tuần tự
2.3. Tính khả tuần tự

2.1. Khái niệm

- **Giá trị A, B ban đầu là 1000 và 2000**
- T_1: Chuyển 50 từ tài khoản A sang tài khoản B.

```
Read(A)
A=A-50
Write(A)
Read(B)
B=B+50
Write(B)
```

- T_2: Chuyển 10% số dư từ tài khoản A sang tài khoản B.

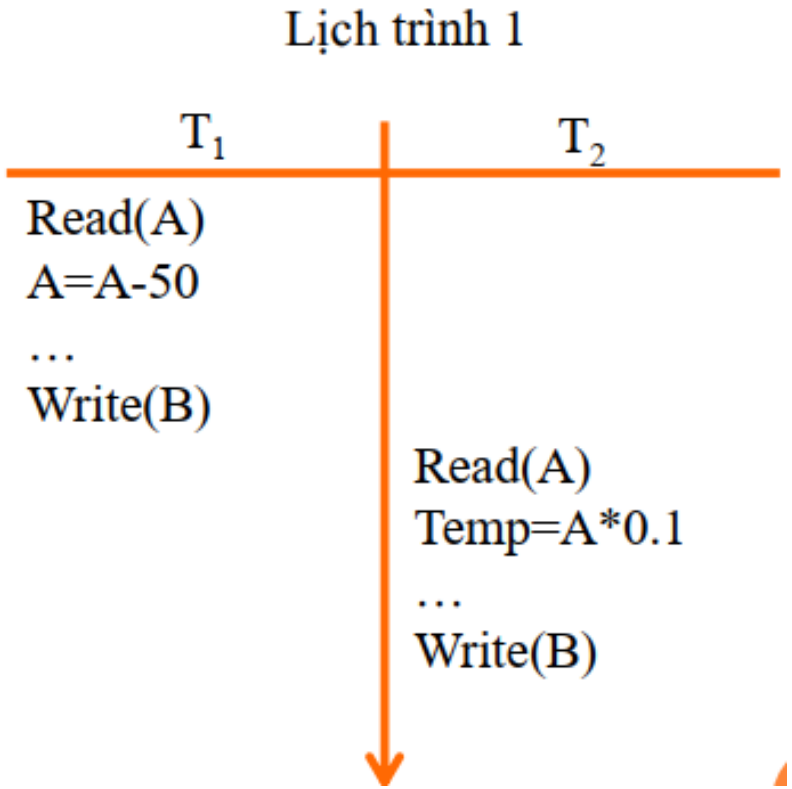
```
Read(A)
Temp=A*0.1
A=A-Temp
Write(A)
Read(B)
B=B+Temp
Write(B)
```

2.1. Khái niệm

- Giả sử 2 giao dịch này được thực hiện theo thứ tự T_1 rồi đến T_2

$\Longrightarrow$ Lịch trình 1 $\rightarrow$ Kết quả: A, B?

A+B trước và sau giao dịch?



- Giả sử 2 giao dịch này được thực hiện theo thứ tự T_2 rồi đến T_1

$\Longrightarrow$ Lịch trình ?$\rightarrow$ Kết quả: A, B?

2.1. Khái niệm

- **Lịch trình**: biểu diễn trình tự thời gian các chỉ thị được thực hiện trong hệ thống.

2.2. Lịch trình tuần tự

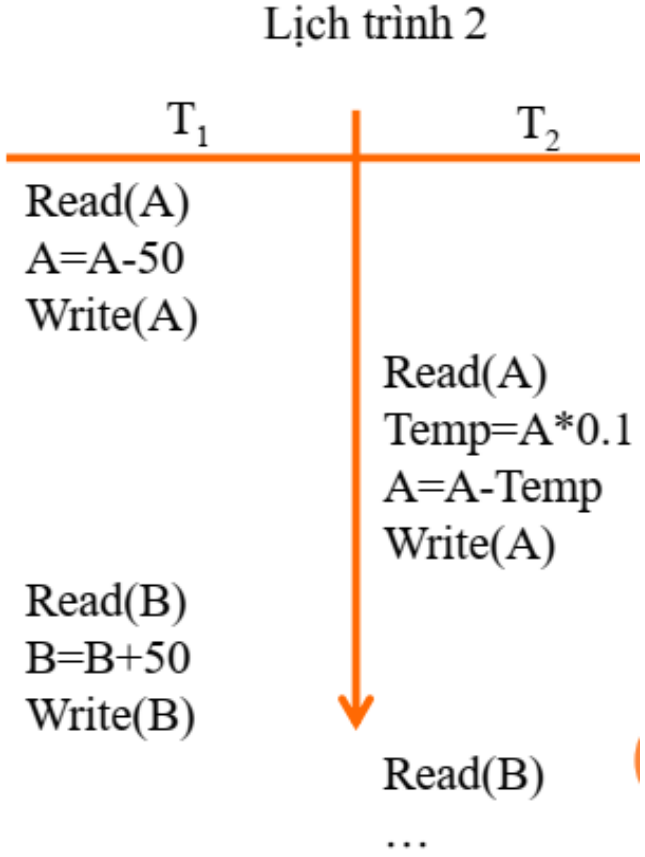
2.2. Lịch trình tuần tự

- Khái niệm: Là lịch trình mà các chỉ thị được thực hiện tuần tự đến khi hoàn tất một giao dịch thì mới chuyển sang thực hiện chỉ thị của giao dịch khác (Không có chỉ thị của giao dịch khác chen vào).
- N giao dịch → N! lịch trình tuần tự hợp lệ khác nhau.

2.2. Lịch trình tuần tự

Lịch trình có xử lý đồng thời

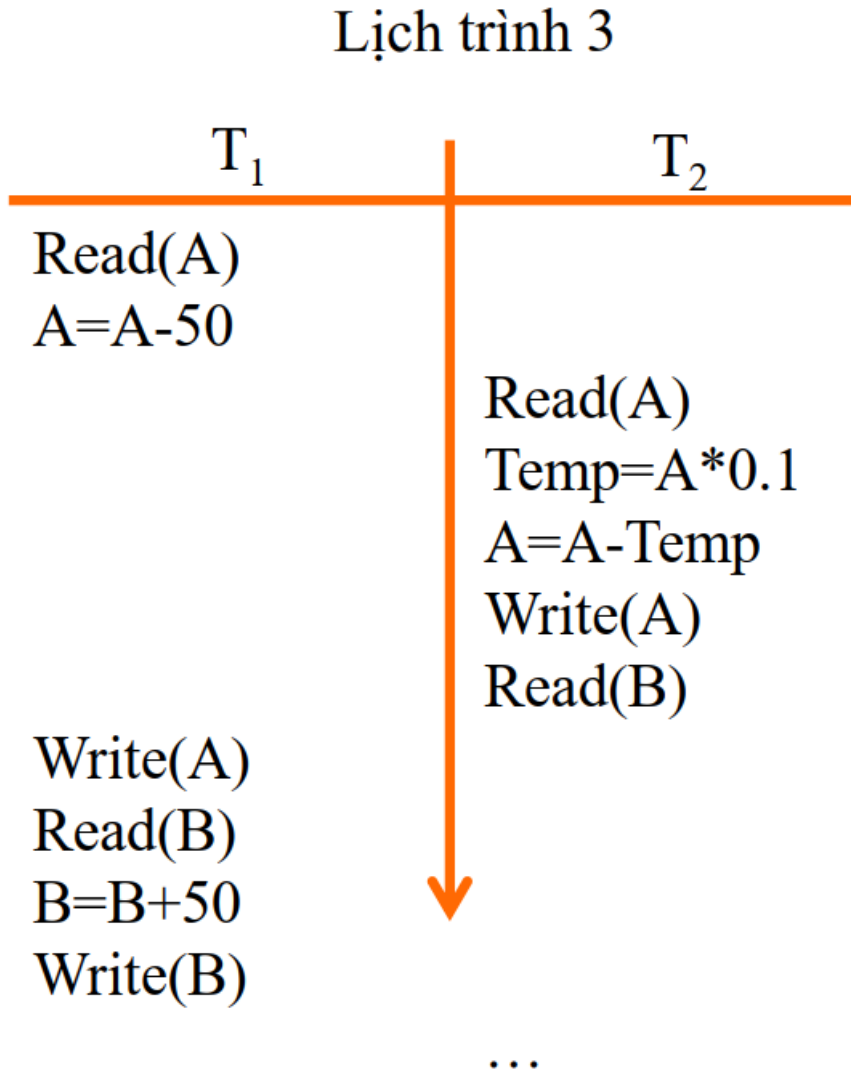
- Trên thực tế (chạy đồng thời): số lịch trình có thể như thế nào với N! ???
- Ví dụ:



2.2. Lịch trình tuần tự

Lịch trình có xử lý đồng thời

- Tất cả lịch trình có xử lý đồng thời đều tương đương nhau???
- Kết quả: A, B?
- Trước và sau giao dịch A+B?



⟹ Để HĐH xử lý cạnhtranh hay không?

2.2. Lịch trình tuần tự

Lịch trình có xử lý đồng thời

- Nhiệm vụ của hệ quản trị CSDL là đảm bảo tính nhất quán của CSDL khi xử lý giao dịch đồng thời.
- Nhiệm vụ của thành phần điều khiển cạnh tranh (concurrency control component).
- Đảm bảo sự nhất quán của CSDL bằng cách **đảm bảo lịch trình được thực hiện có hiệu quả như một lịch trình tuần tự**.

2.3. Tính khả tuần tự

2.3. Tính khả tuần tự

- Tính khả tuần tự (Serializability): Một lịch trình có tính khả tuần tự là một lịch trình **tương đương** với một lịch trình tuần tự nào đó.
- Kết quả tương đương (Result Equivalent): phát sinh cùng trạng thái cuối của CSDL

S_1

read_item(X);
$X := X + 10$;
write_item(X);

S_2
read_item(X); $X := X * 1.1$; write_item (X);

- KQ tương đương là chưa đủ để thể hiện sự tương đương (equivalent) của 2 lịch trình.

2.3. Tính khả tuần tự

Lịch trình tuần tự

???

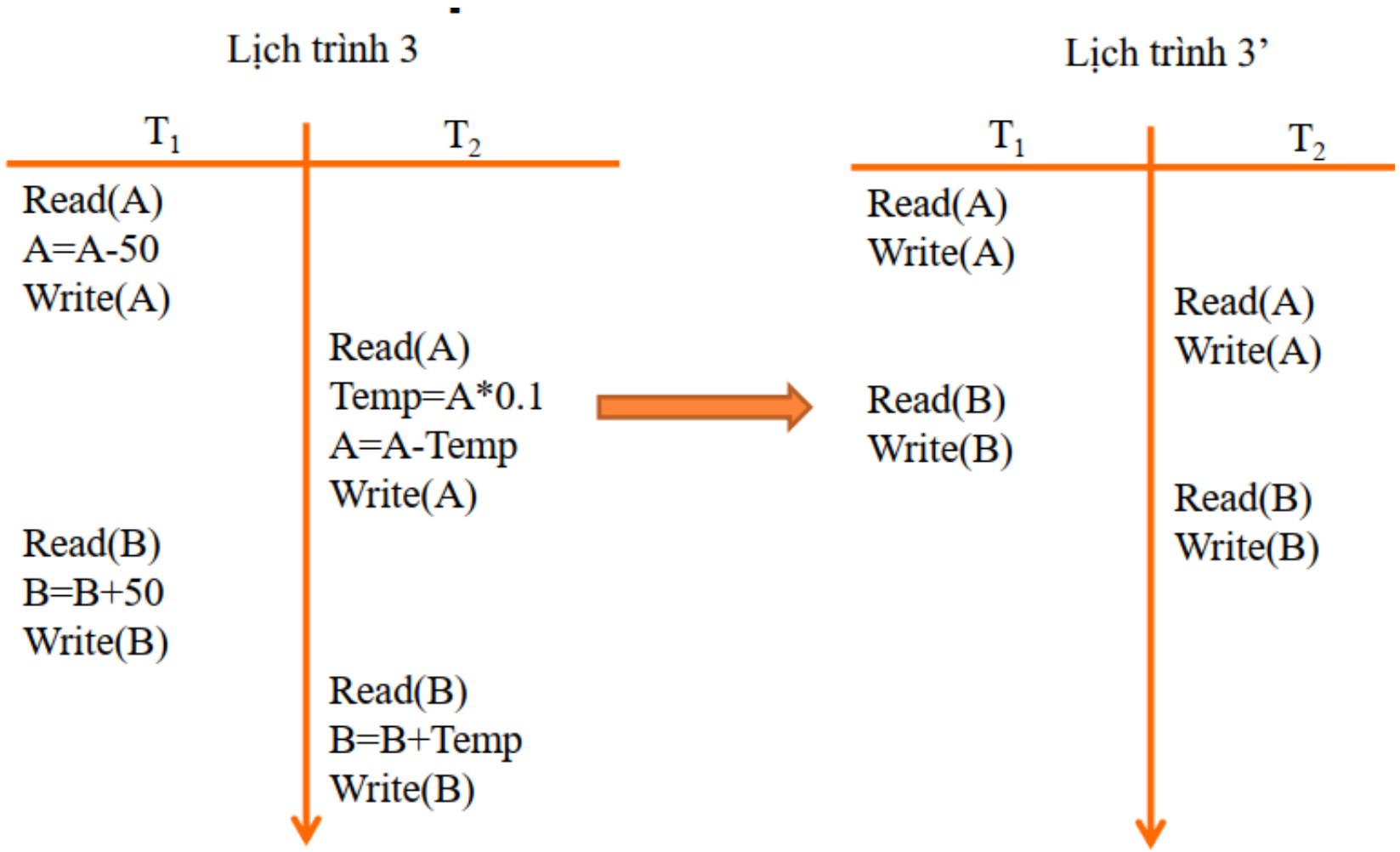
Lịch trình khả tuần tự

2.3. Tính khả tuần tự

- Qui ước
  - Chỉ quan tâm thao tác Read và Write trên các dữ liệu.
  - Giữa Read(X) và Write(X) sẽ có 1 dãy thao tác tùy ý trên bản sao của hạng mục dữ liệu X trong bộ nhớ đệm.

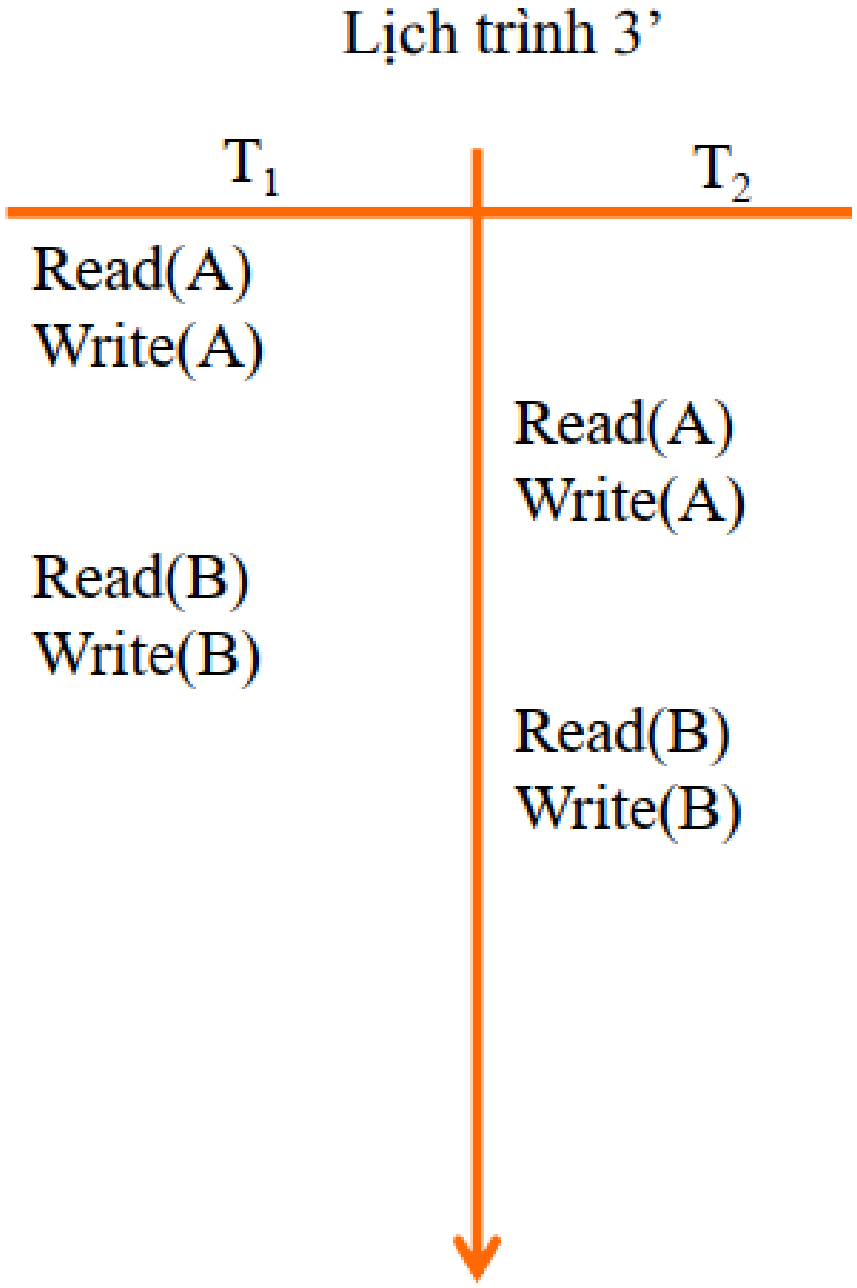
2.3. Tính khả tuần tự

- Biểu diễn lịch trình



2.3. Tính khả tuần tự

- Biểu diễn lịch trình
- S: $R_1(A)\ W_1(A)\ R_2(A)\ W_2(A)$
  $R_1(B)\ W_1(B)\ R_2(B)\ W_2(B)$



2.3. Tính khả tuần tự

Xung đột

- Xung đột (Conflict): 2 chỉ thị xung đột khi ta không thể thay đổi **thứ tự thực hiện** của chúng.
- Xét 2 chỉ thị $I_i$và I_j của 2 giao dịch T_i và T_j tương ứng.
  - I_i(**X**) và I_j (**Y**): không xung đột.
  - I_i =Read(X) và I_j =Read(X): đọc cùng giá trị X bất kể thứ tự I_i và I_j.
  - I_i =Read(**X**) và I_j =Write(**X**): xung đột.
  - I_i =Write(**X**) và I_j =Read(**X**): tương tự.
  - I_i =Write(**X**) và I_j =Write(**X**): thứ tự thực hiện sẽ liên quan đến
    - Read(X) tiếp theo sau bị ảnh hưởng
    - Hoặc Trạng thái CSDL bị ảnh hưởng .

2.3. Tính khả tuần tự

Xung đột

- Vậy có thể tổng hợp lại như sau: 2 chỉ thị trong một lịch trình gọi là xung đột khi nó thỏa 3 điều kiện sau:
  1. Thuộc về 2 giao dịch khác nhau.
  2. Thực hiện trên cùng dữ liệu X.
  3. Ít nhất một chỉ thị là Write(X).

2.3. Tính khả tuần tự

Xung đột

- Ví dụ:

1 – 2: Không xét

2 – 3: Yes

4 – 5: No

6 – 7: Yes

Lịch trình 3’

	T_1	T_2
1.	Read(A)	
2.	Write(A)	
3.		Read(A)
4.		Write(A)
5.	Read(B)	
6.	Write(B)	
7.		Read(B)
8.		Write(B)

2.3. Tính khả tuần tự

Tương đương xung đột

- 2 lịch trình gọi là tương đương về mặt xung đột (conflict equivalent) nếu thứ tự của 2 chỉ thị xung đột giống nhau trong cả 2 lịch trình.
- VD:
  - S1: R1(X), W2(X) và S2: W2(X), R1(X): không tương đương xung đột.

2.3. Tính khả tuần tự

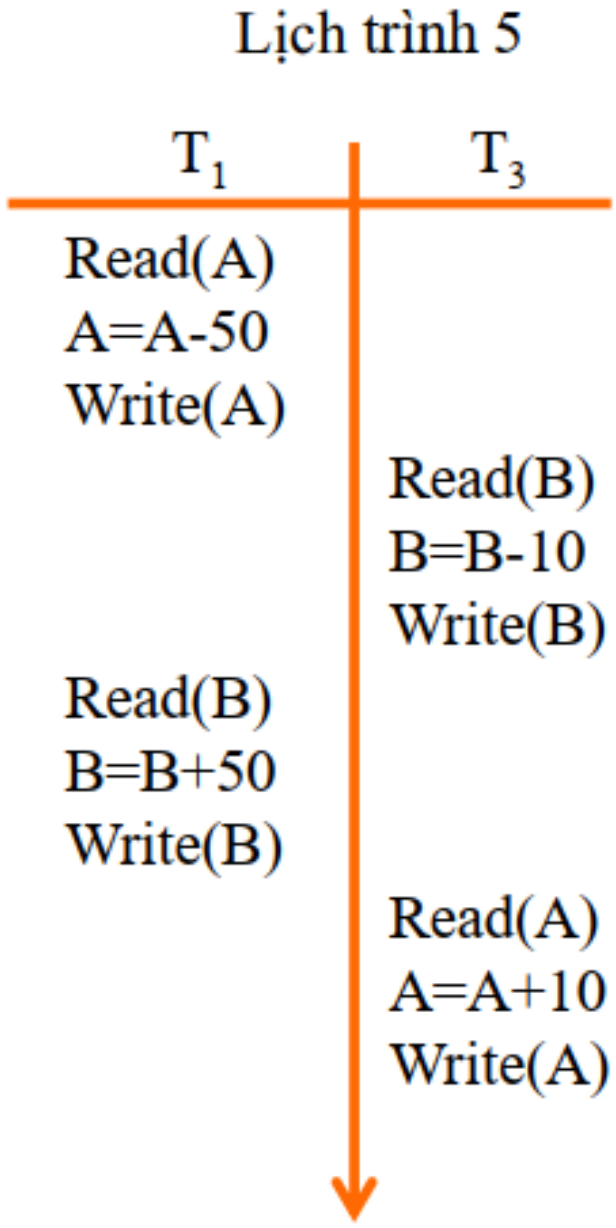
Khả tuần tự xung đột

- Một lịch trình S gọi là khả tuần tự xung đột (conflict serializable) nếu nó tương đương xung đột với 1 lịch trình tuần tự S’ nào đó.
- Khi đó ta có thể thay đổi thứ tự các chỉ thị không xung đột trong S cho đến khi ta được một lịch trình tuần tự S’ tương đương S.

2.3. Tính khả tuần tự

Khả tuần tự xung đột

- Chú ý:
  - A=1000, B=2000
  - Xét T2 là giao dịch chuyển 10 từ B sang A.
  - Kết quả A, B?
  - Lịch trình 5 có khả tuần tự xung đột hay không?



2.3. Tính khả tuần tự

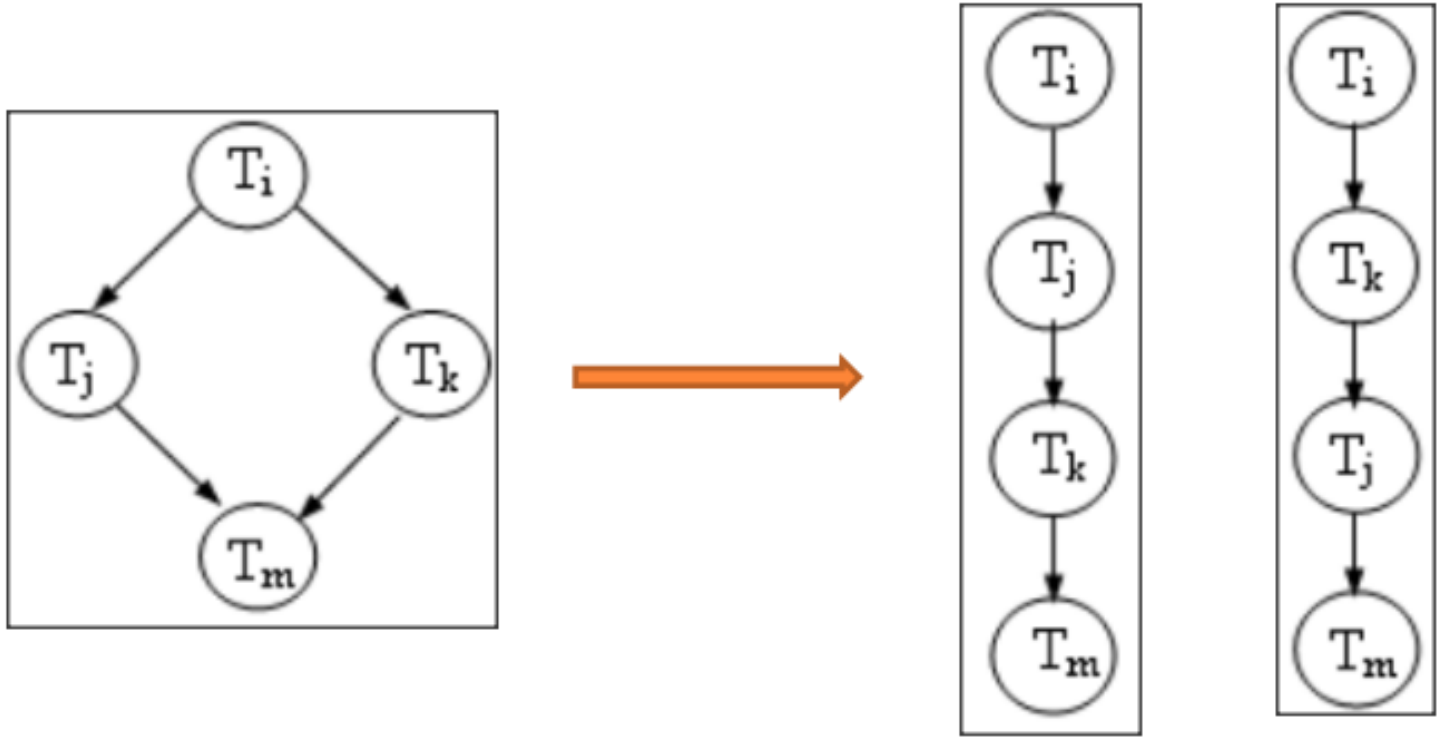
Kiểm tra khả tuần tự xung đột

- Xây dựng một đồ thị có hướng, gọi là đồ thị trình tự (precedence graph) từ lịch trình S cần kiểm tra.
- G=(N, E):
  - N={T1, T2, … , Tn}
  - E là tập hợp các cung.
- Tạo 1 cung $T_i \rightarrow T_j$ khi:
  - T_i thực hiện Write(X) sau đó T_j thực hiện Read(X).
  - T_i thực hiện Read(X) sau đó T_j thực hiện Write(X).
  - T_i thực hiện Write(X) sau đó T_j thực hiện Write(X).
- Lịch trình S là khả tuần tự xung đột nếu G không có churình.
- Chu trình?

2.3. Tính khả tuần tự

Thứ tự khả tuần tự

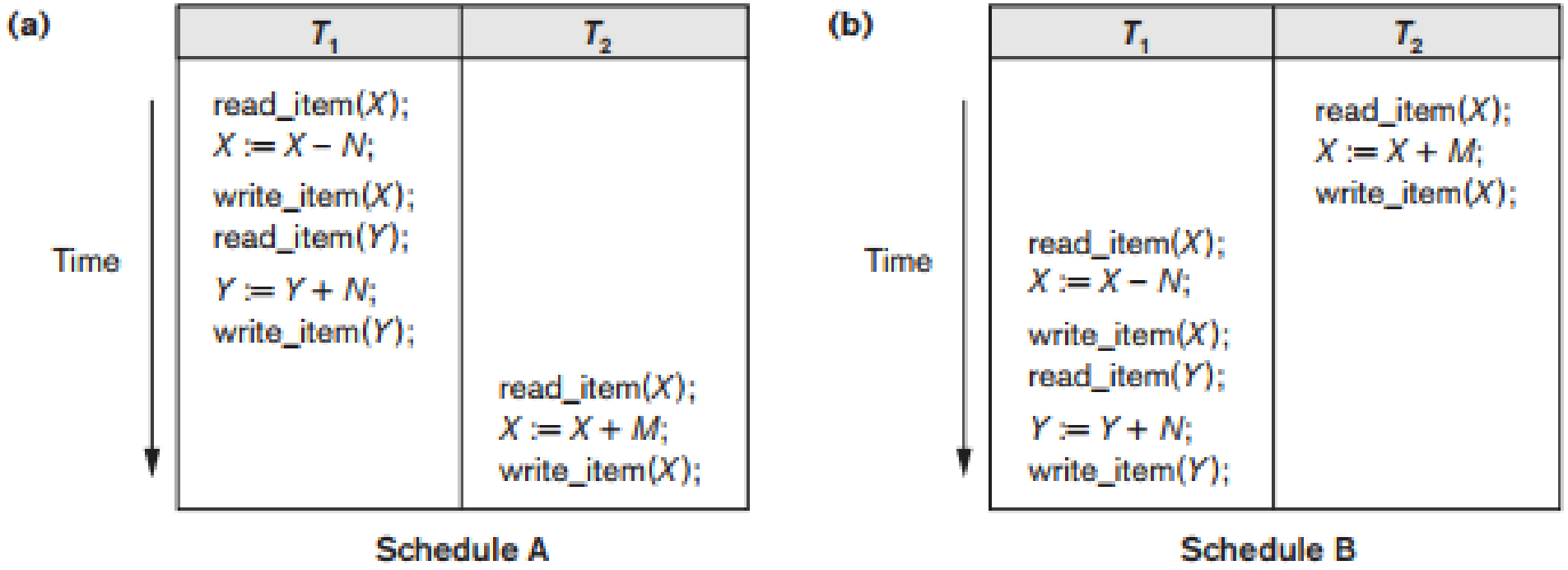
- Thứ tự có thể có được thông qua Topological sorting.



2.3. Tính khả tuần tự

Thứ tự khả tuần tự

- **Ví dụ:**



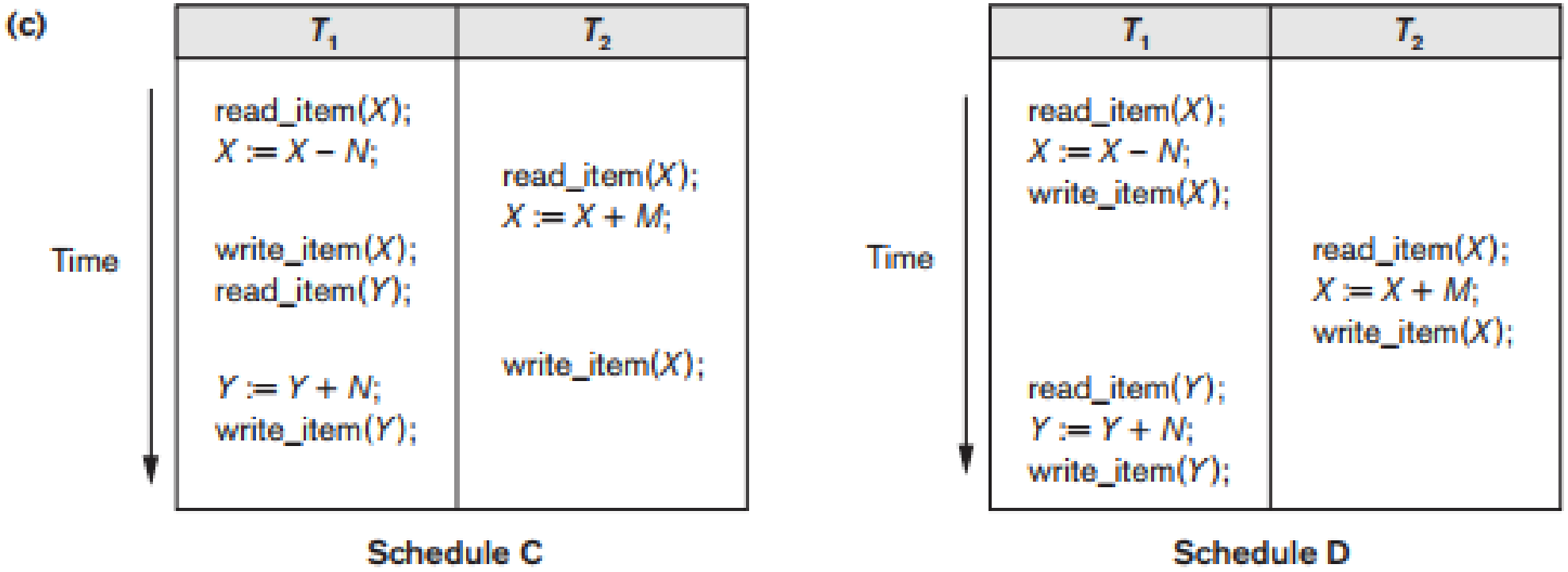
Schedule A

Schedule B

2.3. Tính khả tuần tự

Thứ tự khả tuần tự

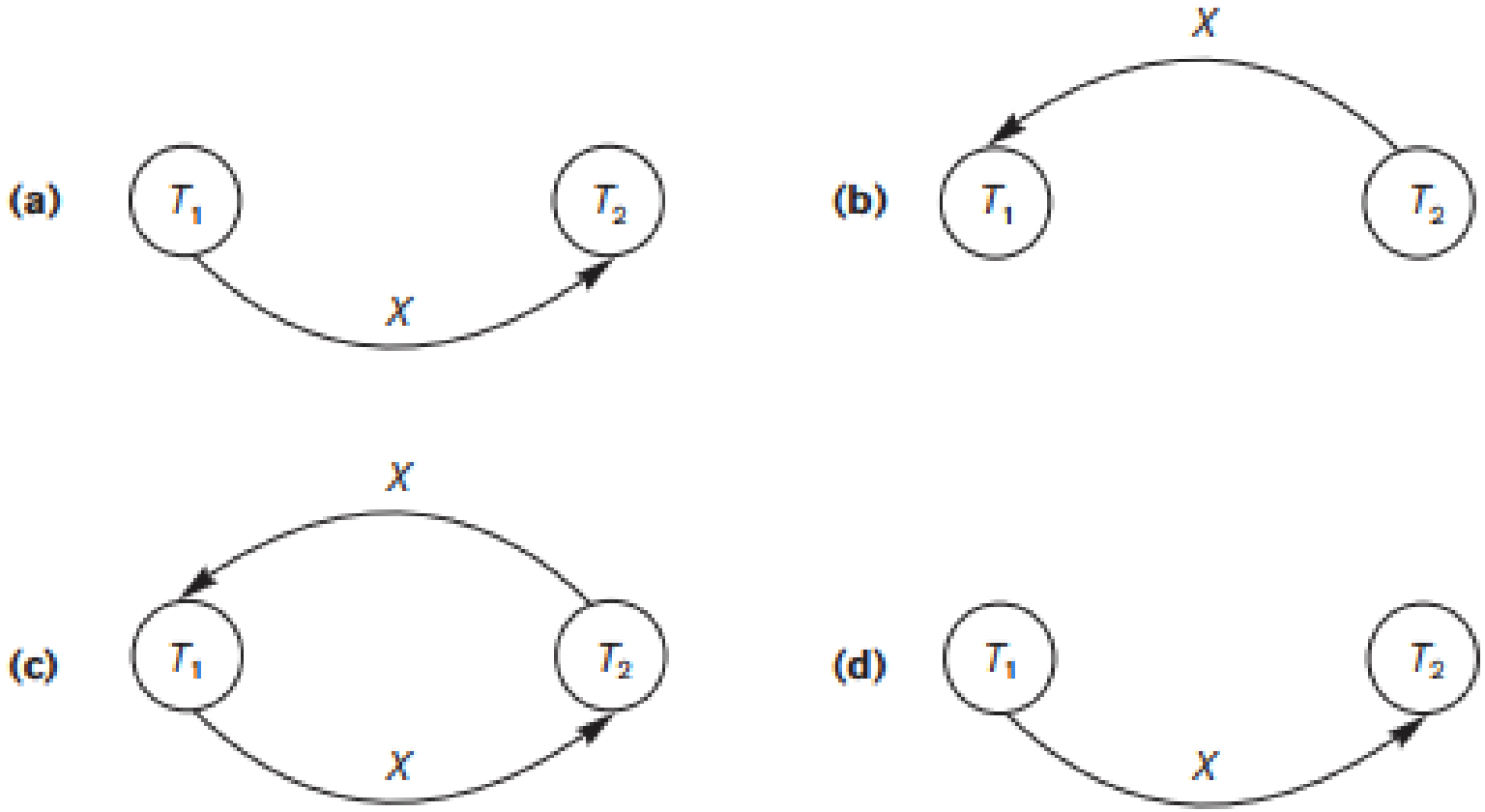
- **Ví dụ:**



2.3. Tính khả tuần tự

Thứ tự khả tuần tự

- **Ví dụ:**



2.3. Tính khả tuần tự

Ứng dụng của khả tuần tự

- Trên thực tế, khó kiểm tra tính khả tuần tự của lịch trình.
- Nếu giao dịch được thực hiện sau đó kết quả của lịch trình được kiểm tra tính khả tuần tự, khi đó ta phải hủy bỏ lịch trình nếu nó không thỏa → Không thực tế.
- Khi giao dịch được gửi liên tục, rất khó kiểm soát khi nào lịch trình bắt đầu và kết thúc.

2.3. Tính khả tuần tự

Bài tập

Lịch biểu nào dưới đây là khả tuần tự?

1. R2(Y), R1(X), R1(Y), R3(X), W3(X), W2(Y), W1(X)
2. R2(Y), R1(X), R1(Y), R3(Z), W3(Z), W2(Y), W1(X)
3. R2(Y), R1(X), R1(Y), R3(X), W2(Y), W1(X), W3(X)
4. R1(X), R1(Y), R3(X), R2(Y), W2(Y), W1(X), W3(X)

Time

Transaction T_1	Transaction T_2	Transaction T_3
	read_item(Z);	
	read_item(Y);	
	write_item(Y);	
		read_item(Y);
		read_item(Z);
read_item(X);		
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(X);	
read_item(Y);		
write_item(Y);		
	write_item(X);	

Schedule E

Time

Transaction T_1	Transaction T_2	Transaction T_3
		read_item(Y);
		read_item(Z);
read_item(X);		
write_item(X);		
		write_item(Y);
		write_item(Z);
	read_item(Z);	
read_item(Y);		
write_item(Y);		
	read_item(Y);	
	write_item(Y);	
	read_item(X);	
	write_item(X);	

Schedule F

2.3. Tính khả tuần tự

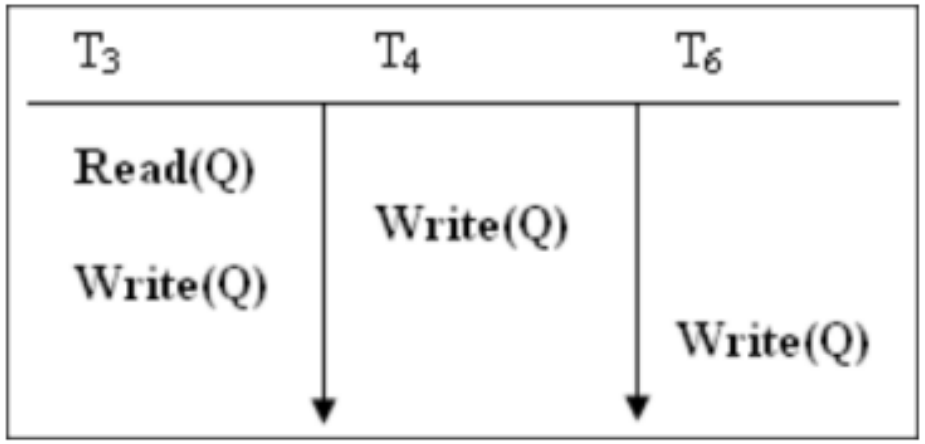
Tương đương View

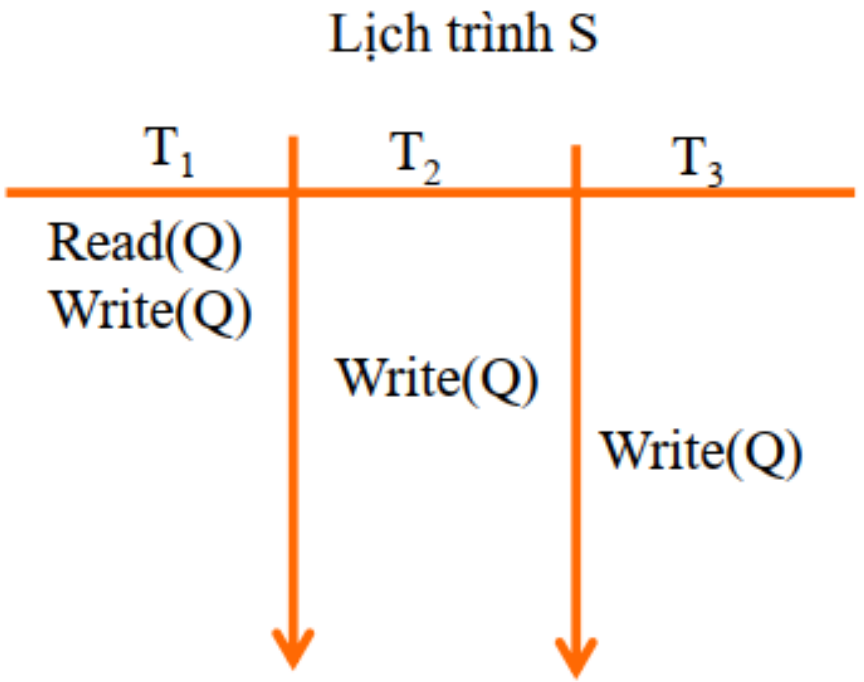
- 2 lịch trình S và S' được gọi là tương đương view (view equivalent) nếu thỏa:
  - Có cùng tập hợp các giao dịch.
  - Với mỗi dữ liệu X, nếu T_i thực hiện Read(X) trong S và giá trị X được sinh ra bởi giao dịch T_j thì T_i cũng phải đọc giá trị X được sinh ra bởi T_j trong S'.
  - Với mỗi dữ liệu X, giao dịch thực hiện Write(X) sau cùng trong S cũng phải thực hiện Write(X) cuối cùng trong S'.
- Các điều kiện đảm bảo:
  - Mỗi giao dịch đọc cùng một giá trị trong cả 2 lịch trình.
  - Cả 2 lịch trình cho ra kết quả trạng thái hệ thống như nhau.
- Ngoài ra còn 1 số loại tương đương khác do các ứng dụng khác nhau đưa ra.

2.3. Tính khả tuần tự

Khả tuần tự View

- Khả tuần tự view (view serializable): 1 lịch trình gọi là khả tuần tự view nếu nó tương đương view với 1 lịch trình tuần tự.





2.3. Tính khả tuần tự

Khả tuần tự View

- Lịch trình khả tuần tự xung đột là khả tuần tự view.
- Lịch trình khả tuần tự view có thể **không** khả tuần tự xung đột
  → Do các write mù (blind write).
- Write mù: thực hiện hoạt động write mà không thực hiện read.

3. Giao dịch trong SQL

3. Giao dịch trong SQL

- Chuẩn SQL đặc tả sự bắt đầu của giao dịch một cách không tường minh.
- Giao dịch được kết thúc bởi một trong 2 lệnh:
  - Commit tran
  - Rollback tran
- Cho phép định nghĩa giao dịch được thực hiện không khả tuần tự.

3. Giao dịch trong SQL

Các dạng READ trong SQL

- **Dirty read**: Giao dịch T1 được phép đọc những thay đổi của T2 chưa được committed. Nếu T2 bị hủy bỏ, giá trị đọc được của T1 không tồn tại và không chính xác.
- **Non-repeatable read**: Giao dịch T1 đọc 1 giá trị từ bảng, nếu có giao dịch T2 thay đổi giá trị đó và T1 đọc lại, T1 sẽ thấy giá trị được cập nhật.
- **Phantoms**: Giao dịch T1 đọc 1 số dòng trong bảng dựa trên 1 số điều kiện Where nào đó. Khi đó 1 giao dịch T2 khác thêm 1 dòng mới thỏa mãn điều kiện Where của T1 vào bảng đó. Nếu T1 đọc lại giá trị của bảng sẽ thấy 1 bóng ma (phantom) – 1 dòng mới được thêm vào, trước đó chưa xuất hiện.

3. Giao dịch trong SQL

Các mức nhất quán trong SQL

- Serializable: mặc định, với nghĩa không cho phép:
  - Dirty read
  - Non-repeatable read
  - Phantoms
- Repeatable read: cho phép Phantoms
- Read committed: chỉ không cho phép Dirty read.
- Read uncommitted: cho phép tất cả.

3. Giao dịch trong SQL

Các mức nhất quán trong SQL

Isolation level	Dirty Reads	Non-repeatable reads	Phantom reads	Concurrency control
READ UNCOMMITTED	Yes	Yes	Yes	Pessimistic
READ COMMITTED (with locking)	No	Yes	Yes	Pessimistic
READ COMMITTED (with snapshot)	No	Yes	Yes	Optimistic
REPEATABLE READ	No	No	Yes	Pessimistic
SNAPSHOT	No	No	No	Optimistic
SERIALIZABLE	No	No	No	Pessimistic

3. Giao dịch trong SQL

- **Cú pháp trong SQL Server**

```
BEGIN { TRAN | TRANSACTION }
      [ { transaction_name | @tran_name_variable }
      [ WITH MARK [ 'description' ] ]
      ]
      [ ; ]
```

- Trong đó:
  - **BEGIN TRAN / BEGIN TRANSACTION**: bắt đầu một giao tác.
  - **transaction_name**: tên của giao tác.
  - **@tran_name_variable**: biến chứa tên giao tác.
  - **WITH MARK**: đánh dấu giao tác trong transaction log.
  - **description**: mô tả cho dấu đánh dấu trong log.
- **Ví dụ: BEGIN** TRAN T1;
  - Giao tác được bắt đầu với tên **T1**.

3. Giao dịch trong SQL

- **Ví dụ sử dụng giao dịch**

```
BEGIN TRAN T2;
SELECT *
FROM NHANVIEN;
COMMIT TRAN;
```

- Trong ví dụ trên:
  - BEGIN TRAN T2: bắt đầu giao tác có tên T2.
  - SELECT * FROM NHANVIEN: thực hiện truy vấn dữ liệu.
  - COMMIT TRAN: xác nhận và hoàn tất giao tác.
- **Xem các thiết lập hiện tại của phiên làm việc:** DBCC USEROPTIONS;
  - Lệnh này cho phép kiểm tra các tùy chọn hiện tại của phiên làm việc, trong đó có **transaction isolation level**.

3. Giao dịch trong SQL

Các mức cô lập giao dịch

- SQL Server cung cấp các mức cô lập giao tác nhằm kiểm soát cách các giao tác đồng thời đọc và ghi dữ liệu.
- Các mức thường sử dụng:
  - READ UNCOMMITTED
  - READ COMMITTED
  - REPEATABLE READ
  - SERIALIZABLE

3. Giao dịch trong SQL

Các mức cô lập giao dịch: READ UNCOMMITTED

- READ UNCOMMITTED cho phép một giao tác **đọc cả những dữ liệu đang được một giao tác khác thay đổi nhưng chưa COMMIT**.
- Đây là **mức cô lập thấp nhất**.
- Mức này có thể xảy ra cả ba hiện tượng đọc không nhất quán:
  - **Dirty Read** – đọc dữ liệu chưa được xác nhận.
  - **Non-repeatable Read** – cùng một dữ liệu nhưng đọc hai lần cho kết quả khác nhau.
  - **Phantom Read** – số lượng bản ghi đọc được thay đổi giữa các lần truy vấn.
- **Cú pháp:**

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;
```

3. Giao dịch trong SQL

Các mức cô lập giao dịch: READ UNCOMMITTED

Giao tác T1

```
BEGIN TRAN T1;

INSERT INTO NHANVIEN
VALUES (
    'NV10', 'A', 'B', 'C', '1999-1-22',
    'SU VAN HANH', 'NAM', 88888, 'P1'
);

WAITFOR DELAY '00:00:10';

ROLLBACK TRAN T1;
```

Giao tác T1 thêm nhân viên NV10 nhưng **chưa COMMIT**.

Giao tác T2

```
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

BEGIN TRAN T2;

SELECT *
FROM NHANVIEN;

COMMIT TRAN T2;
```

Giao tác T2 có thể đọc được bản ghi NV10 mặc dù giao tác T1 chưa hoàn tất.

Nếu T1 thực hiện ROLLBACK, bản ghi NV10 sẽ bị hủy. Khi đó, T2 đã đọc phải dữ liệu chưa được xác nhận → **Dirty Read**.

3. Giao dịch trong SQL

Các mức cô lập giao dịch: READ COMMITTED

- READ COMMITTED là **mức cô lập mặc định của SQL Server**.
- Ở mức này:
  - Chỉ đọc được dữ liệu đã được COMMIT.
  - Không đọc được dữ liệu đang được giao tác khác thay đổi nhưng chưa hoàn tất.
  - SQL Server sử dụng cơ chế **locking** để đảm bảo điều này.
- **Cú pháp**

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

- **Đặc điểm**
  - READ COMMITTED ngăn chặn **Dirty Read**, nhưng vẫn có thể xảy ra:
    - **Non-repeatable Read**
    - **Phantom Read**
- Ví dụ, một giao tác đọc cùng một dòng hai lần; giữa hai lần đọc, giao tác khác cập nhật và COMMIT dòng đó. Kết quả của hai lần đọc có thể khác nhau.

3. Giao dịch trong SQL

Các mức cô lập giao dịch: REPEATABLE READ

- Trong một giao tác, nếu cùng một dữ liệu được truy xuất nhiều lần thì kết quả cần phải **nhất quán**.
- Nếu giao tác khác cập nhật dữ liệu đang được giao tác hiện tại đọc, kết quả của lần đọc sau có thể khác lần đọc trước.
- Để hạn chế hiện tượng này, có thể sử dụng mức cô lập: REPEATABLE **READ**
- Ở mức này, các dòng đã được giao tác đọc sẽ được giữ khóa phù hợp cho đến khi giao tác kết thúc.
- **Cú pháp SET TRANSACTION ISOLATION LEVEL** REPEATABLE **READ**;
- **Đặc điểm REPEATABLE READ:**
  - Ngăn chặn **Dirty Read**.
  - Ngăn chặn **Non-repeatable Read**.
  - Tuy nhiên, vẫn có thể xảy ra **Phantom Read**.

3. Giao dịch trong SQL

Các mức cô lập giao dịch: REPEATABLE READ

- **Thiết lập mức cô lập**

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

GO

DBCC USEROPTIONS;
```

3. Giao dịch trong SQL

Các mức cô lập giao dịch: REPEATABLE READ

- **Giao tác đọc dữ liệu**

```
BEGIN TRAN REPEATREAD;
SELECT *
FROM NHANVIEN;
WAITFOR DELAY '00:00:05’;
SELECT *
FROM NHANVIEN;
COMMIT TRAN REPEATREAD;
```

- Giao tác REPEATREAD thực hiện hai lần SELECT cách nhau 5 giây.
- Trong thời gian này, nếu một giao tác khác cố gắng cập nhật một dòng mà REPEATREAD đã đọc, giao tác cập nhật có thể phải **chờ** cho đến khi giao tác REPEATREAD kết thúc.

3. Giao dịch trong SQL

Các mức cô lập giao dịch: REPEATABLE READ

- **Giao tác cập nhật**

```
BEGIN TRAN SHORTUPDATE;

UPDATE NHANVIEN

SET HONV = 'A’

WHERE MANV = 'NV10’;

COMMIT TRAN SHORTUPDATE;
```

- Nếu NV10 đang bị giao tác REPEATREAD giữ khóa đọc, giao tác SHORT UPDATE có thể phải chờ.

3. Giao dịch trong SQL

Các mức cô lập giao dịch: SERIALIZABLE

- SERIALIZABLE là **mức cô lập cao nhất** trong các mức cô lập truyền thống của SQL Server.
- Mục tiêu là đảm bảo kết quả của các giao tác đồng thời tương đương với trường hợp các giao tác được thực hiện **tuần tự**.
- **Cú pháp:** `SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;`
- Ở mức này, SQL Server sử dụng các khóa chặt chẽ hơn để ngăn các giao tác khác:
  - Đọc dữ liệu chưa COMMIT.
  - Thay đổi dữ liệu mà giao tác hiện tại đang đọc.
  - Chèn hoặc xóa các bản ghi làm thay đổi tập kết quả của truy vấn.
- Do đó, SERIALIZABLE ngăn chặn:
  - **Dirty Read**
  - **Non-repeatable Read**
  - **Phantom Read**
- Đổi lại, mức cô lập này có thể làm **giảm khả năng đồng thời (concurrency)** và tăng nguy cơ **blocking/deadlock**.

3. Giao dịch trong SQL

Các mức cô lập giao dịch

- Có thể thay đổi mức cô lập của phiên làm việc bằng lệnh:
  - **SET TRANSACTION ISOLATION LEVEL SERIALIZABLE**;
- Hoặc:
  - **SET TRANSACTION ISOLATION LEVEL READ** UNCOMMITTED;
- Hoặc:
  - **SET TRANSACTION ISOLATION LEVEL READ COMMITTED**;
- Hoặc:
  - **SET TRANSACTION ISOLATION LEVEL** REPEATABLE **READ**;
- Có thể kiểm tra thiết lập hiện tại bằng:
  - DBCC USEROPTIONS;

3. Giao dịch trong SQL

- **So sánh các mức cô lập**

Mức cô lập	Dirty Read	Non-repeatable Read	Phantom Read	Mức độ cô lập
READ UNCOMMITTED	Có	Có	Có	Thấp nhất
READ COMMITTED	Không	Có	Có	Thấp
REPEATABLE READ	Không	Không	Có	Cao
SERIALIZABLE	Không	Không	Không	Cao nhất

3. Giao dịch trong SQL

- **Thứ tự từ thấp đến cao**

READ UNCOMMITTED

↓

READ COMMITTED

↓

REPEATABLE READ

↓

SERIALIZABLE

- **Mức cô lập càng cao → tính nhất quán càng cao nhưng khả năng đồng thời thường càng thấp.**

4. Bài tập

4. Bài tập

- **Bài 1: Viết chương trình:**

Transaction T_1
read_item(X);
write_item(X);
read_item(Y);
write_item(Y);

Transaction T_2
read_item(Z);
read_item(Y);
write_item(Y);
read_item(X);
write_item(X);

Transaction T_3
read_item(Y);
read_item(Z);
write_item(Y);
write_item(Z);

- Sinh tất cả các schedule có thể của ba transaction.
- Xác định schedule nào conflict serializable và schedule nào không.
- Với mỗi schedule conflict serializable:
  - In schedule.
  - Liệt kê các serial schedule tương đương.

4. Bài tập

- **Bài 2.** Thêm thao tác commit vào cuối mỗi transaction T1 và T2.
  - Liệt kê tất cả các schedule có thể xảy ra.

(a)

T_1

read_item(X);
$X := X - N$;
write_item(X);
read_item(Y);
$Y := Y + N$;
write_item(Y);

(b)

T_2

read_item(X);
$X := X + M$;
write_item(X);

4. Bài tập

- **Bài 3.**
- Cho hai giao dịch cùng thao tác trên các mục dữ liệu P và Q của một hệ thống quản lý kho hàng:

  T1: Đọc(P), Ghi(Q), Đọc(Q)

  T2: Ghi(P), Đọc(Q), Ghi(Q)
- Thực hiện các yêu cầu sau:
  - Liệt kê tất cả các cặp thao tác giữa T1 và T2 có khả năng xung đột (conflict).
  - Với mỗi cặp xung đột, cho biết đó là loại xung đột nào (đọc-ghi, ghi-đọc, hay ghi-ghi).
  - Xây dựng một lịch thực thi xen kẽ (interleaved schedule) hợp lệ của T1 và T2 mà thứ tự các thao tác xung đột giống như thứ tự trong lịch tuần tự T1 → T2.

4. Bài tập

- **Bài 4.** Một ứng dụng đặt chỗ sân bóng đá có các loại giao dịch sau:
  - (i) Đăng ký một khách hàng mới vào hệ thống.
  - (ii) Đặt một khung giờ sân còn trống cho khách hàng.
  - (iii) In báo cáo tổng số lượt đặt sân trong tuần (chỉ đọc, không cần chính xác tuyệt đối tới từng giây).
  - (iv) Kiểm tra và hủy các đặt chỗ đã quá hạn thanh toán, đồng thời giải phóng khung giờ đó cho khách khác đặt lại.
- Với mỗi loại giao dịch, hãy đề xuất một mức cô lập SQL (READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, hoặc SERIALIZABLE) phù hợp và giải thích lý do lựa chọn, có đề cập đến hiện tượng (dirty read, non-repeatable read, hoặc phantom) mà mức đó ngăn chặn hoặc cho phép xảy ra.

4. Bài tập

- **Bài 5.** Cho lịch thực thi S gồm ba giao dịch trên các mục dữ liệu M, N, K:
  - S: R1(M) R2(K) R3(M) W1(M) R2(M) W3(K) W2(N) W1(N)

(chỉ số dưới i biểu diễn thao tác thuộc giao dịch Ti)

1. Vẽ đồ thị tuần tự hóa (precedence graph / serialization graph) cho lịch S.
2. Lịch S có khả tuần tự hóa xung đột (conflict-serializable) hay không? Nếu có, hãy chỉ ra (các) lịch tuần tự tương đương.
3. Nếu đổi vị trí của W3(K) lên trước R2(K), đồ thị tuần tự hóa thay đổi như thế nào? Kết luận có thay đổi không?

Tài liệu tham khảo

- Raghu Ramarkrishnan, Johannes Gehrke. *Database Management System*. 3rd Edition, Mc Graw Hill, 2003.
- Tutorials Point, Database Management System, www.tutorialspoint.com, 2015.
- Hector Garcia-Molina, Jeffrey D. Ullman and Jennifer Widom. *Database Systems, the Complete Book*, 2nd Edition, Prentice Hall, 2009
- Ramez Elmasri, Shamkant B. Navathe. *Fundamentals of Database Systems*. *7th Edition*, Pearson, 2015
- Silberschatz,Korth and Sudarshan, Database System Concepts (6th Edition), McGraw-Hill, 2010.

Hết!
Cảm ơn Anh/ Chị đã theo dõi!

GVHD: TS. Phan Thị Thể

FIT – HCM-UTE

Email: thept@hcmute.edu.vn